МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 6

Тема занятия управление табличными структурами dataframe в rpa
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: сформировать у студентов устойчивые практические навыки создания и трансформации табличных структур DataFrame в RPA для решения прикладных задач обработки и анализа данных.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Основные понятия

В современных RPA-проектах робот редко работает только с отдельными переменными. Чаще всего он взаимодействует с массивами данных: выгрузками из Excel, Google Sheets, CRM-систем, бухгалтерских программ, API-сервисов.

Для эффективной обработки таких данных используется структура DataFrame – табличная модель хранения информации, состоящая из строк и столбцов.

Если представить переменную как одну ячейку, список – как один столбец, то *DataFrame* – это полноценная таблица с логической структурой:

- строки – записи (клиенты, заказы, сотрудники);
- столбцы – характеристики (имя, дата, сумма, статус);
- индексы – техническая нумерация строк;
- типы данных – текст, число, дата, логическое значение.

В RPA *DataFrame* применяется как основной инструмент для:

- обработки выгрузок;
- объединения данных из разных источников;
- фильтрации информации;
- расчётов и агрегации;
- подготовки итоговых отчетов.

Работа с *DataFrame* приближает разработчика RPA к уровню аналитической обработки данных и позволяет строить сложные сценарии без использования внешних инструментов.

После изучения темы вы сможете:

- Понимать архитектуру и логику работы структуры *DataFrame* в *Puzzle RPA*
- Самостоятельно создавать *DataFrame* из различных источников данных
- Добавлять строки и столбцы с соблюдением структуры и типов данных
- Объединять таблицы по ключам с учетом бизнес-логики процесса
- Корректно добавлять таблицы по строкам и по столбцам
- Преобразовывать *DataFrame* в словарь и другие форматы для передачи в функции и формы
- Применять фильтрацию для отбора нужных записей
- Выполнять группировку данных и рассчитывать агрегированные показатели
- Строить аналитическую логику внутри RPA-процесса без использования внешних инструментов
- Проектировать устойчивые автоматизации, основанные на обработке табличных данных

Вы научитесь избегать типичных ошибок:

- Несоответствие структуры при добавлении строк
- Ошибки типов данных (текст вместо числа, строка вместо даты)
- Потеря данных при объединении таблиц
- Дублирование записей
- Неверный выбор типа объединения (inner/left/right)
- Нарушение логики при добавлении таблиц по строкам и столбцам
- Ошибки фильтрации из-за некорректных условий
- Некорректная группировка без указания агрегирующей функции
- Перезапись переменных и потеря промежуточных данных

В результате вы будете не просто использовать *DataFrame*, а осознанно управлять табличными структурами внутри архитектуры RPA-процесса.

Понятие *DataFrame* в RPA

В классическом понимании робот – это инструмент, который повторяет действия пользователя: нажимает кнопки, вводит текст, скачивает файлы.

Но в продвинутой автоматизации робот – это система обработки данных.

И именно здесь появляется ключевая структура – *DataFrame*. *DataFrame* – это табличная модель хранения данных, представляющая собой упорядоченную структуру строк и столбцов, где каждая колонка имеет имя и тип данных, а каждая строка – логическую запись.

В RPA *DataFrame* – это не просто таблица. Это:

- контейнер бизнес-данных,
- инструмент аналитики,
- механизм принятия решений,
- архитектурная основа процесса.

Если переменные – это отдельные значения, если списки – это линейная последовательность, то *DataFrame* – это многомерная логическая модель.

Именно поэтому в сложных процессах *DataFrame* становится ядром всей логики. Многие начинающие воспринимают *DataFrame* как «таблицу в памяти». Но есть принципиальная разница между Excel-файлом и *DataFrame*:

Excel	*DataFrame*
Визуальная таблица	Программная структура
Зависит от файла	Работает в памяти
Требует ручной логики	Поддерживает встроенные операции
Ориентирован на пользователя	Ориентирован на обработку

DataFrame позволяет:

- фильтровать данные без циклов,
- объединять таблицы одной операцией,
- рассчитывать агрегаты без ручного подсчета,
- трансформировать данные за один шаг.

В продвинутом RPA-процессе количество циклов можно сократить в 2–3 раза, если грамотно использовать операции *DataFrame* вместо построчной обработки.

Внутренняя логика *DataFrame*

DataFrame состоит из:

- Столбцов (Columns) – логические признаки данных
- Строк (Rows) – отдельные записи
- Индекса (Index) – технической нумерации
- Типов данных (Data Types)

Каждый столбец имеет тип:

- строковый,
- числовой,
- логический,
- дата/время.

Типизация – это фундамент устойчивости процесса. Если изначально правильно задать типы столбцов, можно избежать 70% ошибок при фильтрации и объединении.

Например:

- текстовое "10000" не равно числовому 10000;
- строка "01.02.2024" не равна дате 1 февраля 2024.

В реальных проектах робот работает минимум с двумя источниками данных:

- бухгалтерская система
- CRM
- Excel-отчёт
- Google таблица
- API

Форматы данных отличаются. Названия столбцов отличаются. Типы данных отличаются. *DataFrame* становится универсальным слоем нормализации.

Процесс выглядит так:

- Получили данные
- Привели к единой структуре *DataFrame*
- Очистили
- Обработали
- Отдали результат

Профессиональные RPA-разработчики всегда создают «эталонную структуру *DataFrame*», к которой приводят все входящие данные. Это делает процесс масштабируемым.

Главное отличие продвинутого подхода:

DataFrame используется не только для хранения данных, но и для построения логики.

Через него можно:

- определить, какие строки обрабатывать;
- выявить дубликаты;
- найти расхождения;
- рассчитать показатели;
- определить исключения.

Например:

- Если сумма оплаты меньше суммы счета – добавить в таблицу исключений.
- Если дата просрочена – добавить флаг.
- Если клиент новый – отметить его статус.

Это уже не просто таблица – это интеллектуальный фильтр.

Вместо хранения отдельных списков «успешно», «ошибка», «в работе» можно добавить столбец `status` в *DataFrame* и управлять всем процессом через фильтрацию.

Архитектурная роль DataFrame в процессе

В продвинутой архитектуре RPA процесс часто строится так:

- Блок загрузки данных
- Блок трансформации
- Блок аналитики
- Блок принятия решения
- Блок вывода результата

Во всех этих блоках используется один и тот же *DataFrame*.

Это снижает:

- дублирование логики,
- количество переменных,
- риск потери данных.

Один *DataFrame* может проходить через весь процесс, постепенно «обогащаясь» новыми столбцами и результатами проверок.

Если робот работает с 10 строками – разницы почти нет. Но если данных 10 000? Построчная обработка становится медленной и сложной. *DataFrame*-операции работают быстрее и структурированнее.

Особенно это важно при:

- ежедневных выгрузках,
- финансовых отчётах,
- массовой обработке заказов,
- консолидации данных.

DataFrame позволяет:

- работать пакетно,
- выполнять операции над столбцами,
- использовать группировку,
- объединять таблицы без вложенных циклов.

Новички мыслят так: `Робот взял строку → обработал → пошёл дальше.`

Продвинутый подход: `Робот сформировал DataFrame → проанализировал массив → выделил нужные записи → выполнил действия.`

Это другой уровень мышления.

DataFrame учит:

- мыслить структурами,
- проектировать данные заранее,
- учитывать масштаб,
- отделять бизнес-логику от технической реализации.

Типичные заблуждения о *DataFrame*:

- Это просто таблица
- Его нужно использовать только при больших данных
- Проще работать через циклы
- Excel удобнее

На практике:

- *DataFrame* – это логический инструмент

- Даже 20 строк удобнее фильтровать через структуру
- Он уменьшает код и блоки
- Повышает читаемость проекта

DataFrame в RPA – это:

- структурированный контейнер данных
- инструмент аналитики
- средство объединения источников
- механизм фильтрации
- архитектурная основа процесса

Понимание *DataFrame* – это переход от автоматизации действий к автоматизации обработки данных. Именно поэтому владение *DataFrame* является обязательным навыком разработчика RPA высокого уровня.

Создание *DataFrame* в RPA

Создание *DataFrame* – это не просто технический шаг, а этап проектирования структуры будущего процесса.

Именно в момент создания таблицы закладывается логика обработки данных, их формат, устойчивость и масштабируемость.

Если структура задана правильно – дальнейшая работа упрощается в разы.

Если структура создана хаотично – процесс начинает «сыпаться» на этапе объединений, фильтрации и группировок.

Создать *DataFrame* – значит:

- определить набор столбцов;
- задать их имена;
- определить типы данных;
- сформировать начальную структуру (пустую или заполненную);
- сохранить таблицу в переменную для дальнейшей обработки.

Это момент, когда вы переводите данные из хаотичного набора значений в управляемую модель. В продвинутом подходе *DataFrame* создаётся до начала обработки, а не «по ходу» процесса.

Создание пустого DataFrame с заданной структурой

Это профессиональный подход. Вы заранее определяете столбцы:

- `order_id`
- `client`
- `amount`
- `status`
- `date`

Даже если данных ещё нет, структура уже существует. Зачем это нужно?

- гарантируется одинаковый формат при добавлении строк;
- снижается риск несоответствия ключей;
- упрощается объединение с другими таблицами;

Пустой *DataFrame* с заданной структурой – это «контейнер», в который робот будет складывать обработанные данные.

Создание DataFrame из списка словарей

Если данные поступают в цикле (например, из формы или API), можно формировать список записей, где каждая запись – словарь. Пример логики:

- получаем данные;
- формируем словарь;
- добавляем в список;
- создаём *DataFrame* из списка.

Преимущество:

- удобно работать с динамическими источниками;
- гибкость структуры.

Недостаток:

- риск несовпадения ключей;
- сложнее контролировать типы данных.

Если используете этот способ, заранее фиксируйте перечень ключей – это предотвращает появление «лишних» столбцов.

Создание DataFrame из Excel или Google Sheets

Один из самых частых сценариев.

Робот:

- Загружает файл.
- Получает табличные данные.
- Преобразует их в *DataFrame*.

Важно учитывать:

- есть ли заголовок;
- в какой строке он расположен;
- корректны ли названия столбцов;
- есть ли пустые строки.

После загрузки данных почти всегда требуется этап «очистки структуры»:

- удаление пустых строк,
- переименование столбцов,
- приведение типов.

Иначе дальнейшие операции могут работать некорректно.

Создание DataFrame из другого DataFrame

Иногда создаётся новая таблица:

- как результат фильтрации;
- как результат группировки;
- как объединённая таблица.

Это уже этап трансформации, но по сути – создание нового объекта *DataFrame*. Перед созданием *DataFrame* нужно ответить на вопросы:

- Какие столбцы обязательны?
- Какие будут расчетными?
- Какие данные используются для объединения?
- Какие типы данных нужны?

- Какие столбцы будут участвовать в фильтрации?

Это превращает создание таблицы в архитектурный этап.

Всегда разделяйте:

- входные данные,
- промежуточные данные,
- итоговые данные.

Лучше создавать разные *DataFrame* для разных этапов, чем перегружать один.

Типы данных – критически важный момент

При создании *DataFrame* необходимо учитывать типизацию:

Тип	Где используется
Строка	Имена, статусы, коды
Число	Суммы, количество
Дата	Даты операций
Логический	Флаги проверки

Неправильный тип = потенциальная ошибка в будущем.

Пример проблемы:

- Сумма хранится как текст → фильтр «больше 10000» не работает.
- Дата хранится как строка → сортировка работает некорректно.

Лучше сразу приводить столбцы к нужному типу после создания *DataFrame*, чем исправлять ошибки на этапе аналитики.

Именование столбцов – основа устойчивости

Хорошая практика:

- использовать понятные названия;
- избегать пробелов;
- не использовать спецсимволы;
- придерживаться единого стиля (snake_case или camelCase).

Плохой пример:

- «Сумма оплаты»
- «Дата операции (факт)»
- Хороший пример:
- payment_amount
- operation_date

Это особенно важно при объединении таблиц по ключам.

Частые ошибки при создании *DataFrame*:

- Разные названия столбцов в одинаковых таблицах
- Пробелы в названиях
- Смешанные типы данных в одном столбце
- Отсутствие проверки пустых данных
- Создание таблицы без понимания будущих операций

Создание *DataFrame* – это:

- проектирование модели данных,
- подготовка к аналитике,
- формирование основы логики,
- точка входа для всей обработки.

Если структура создана правильно, дальше процесс становится предсказуемым и логичным.

Если структура создана хаотично, робот будет постоянно «ломаться» при объединении, фильтрации и группировке.

Создание *DataFrame* в Puzzle RPA – это:

- переход от набора переменных к системе;
- основа аналитической автоматизации;
- инструмент масштабируемости;
- фундамент устойчивой архитектуры процесса.

Понимание этого этапа отличает начинающего разработчика от профессионала.

ТАБЛИЧНЫЕ ДАННЫЕ В PUZZLE RPA

В Puzzle RPA табличные данные чаще всего реализуются через структуру *DataFrame*, которая становится рабочей моделью данных внутри процесса.

Важно понимать: робот не просто «кликает» по интерфейсу – он обрабатывает данные. И чем лучше структурированы данные, тем стабильнее и масштабируемее автоматизация.

Работа с табличными данными в Puzzle RPA включает:

- создание таблиц;
- изменение структуры (добавление строк и столбцов);
- объединение данных;
- фильтрацию и группировку;
- преобразование форматов;
- подготовку аналитических результатов.

Освоение этих инструментов переводит разработчика на уровень проектирования процессов, где управление данными становится основой всей логики автоматизации.

В данной методичке мы рассмотрим принципы работы с табличными структурами и научимся применять их для построения устойчивых, профессиональных RPA-решений.

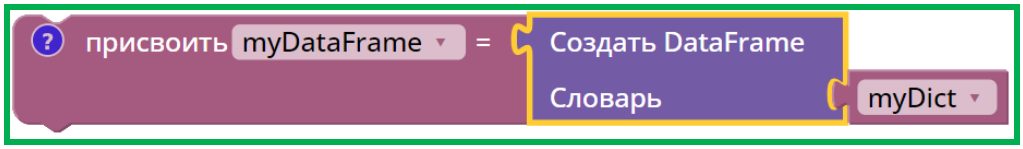
В Puzzle RPA все инструменты для работы с таблицами собраны во вкладке «Стандартные → Табличные данные».

Это отдельная группа блоков, предназначенная для создания, трансформации и аналитической обработки структурированных данных внутри процесса.

Давайте рассмотрим каждый блок подробно и с точки зрения архитектуры применения.

Блок «Создать *DataFrame*»

Блок «Создать *DataFrame*» – это фундамент работы с табличными данными в Puzzle RPA. Блок позволяет создать *DataFrame*. Для этого блоку необходимо передать словарь. Созданный *DataFrame* сохраняется в переменную.



Правильное использование этого блока закладывает основу для всех последующих операций: добавления строк и столбцов, объединения таблиц, фильтрации и группировки.

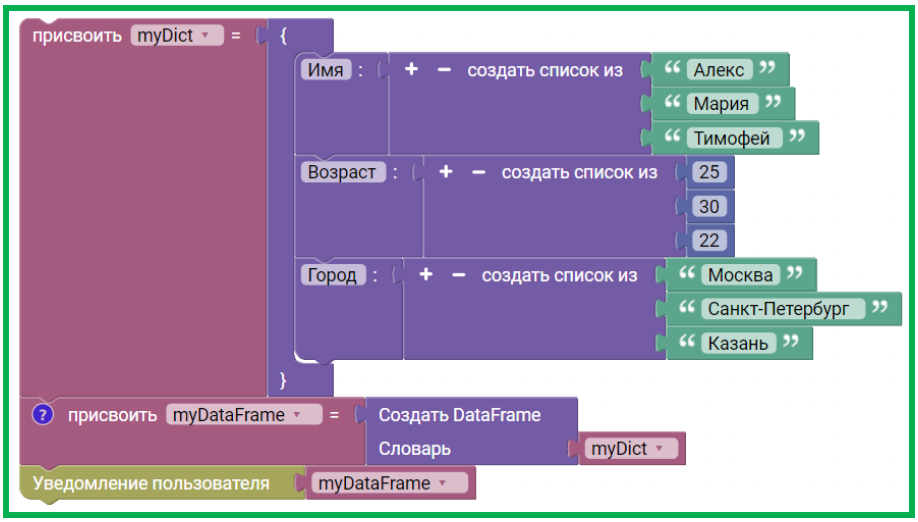
Блок «Создать *DataFrame*» используется для:

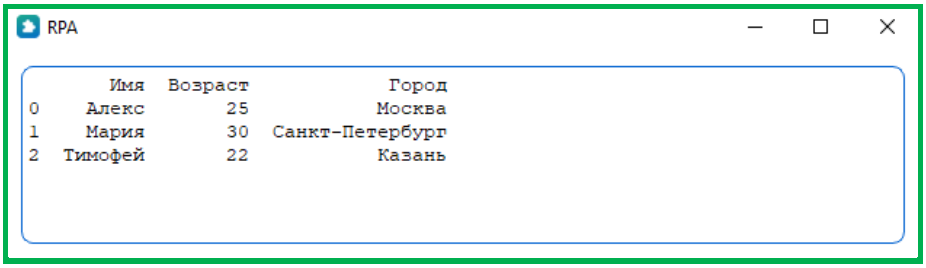
- создания новой табличной структуры для хранения данных;
- подготовки пустого *DataFrame* с заданной структурой;
- централизованного хранения данных для аналитических и операционных действий в процессе.

Блок «Создать *DataFrame*» – это точка входа для работы с табличными данными в Puzzle RPA.

Пример использования.

В данном примере создается словарь с тремя ключами (каждый ключ соответствует столбцу таблицы). В качестве значений в словарях выступают списки, в которых каждое значение соответствует ячейке одной из строк. На основе словаря создается *DataFrame*. Созданный *DataFrame* выводится на экран.



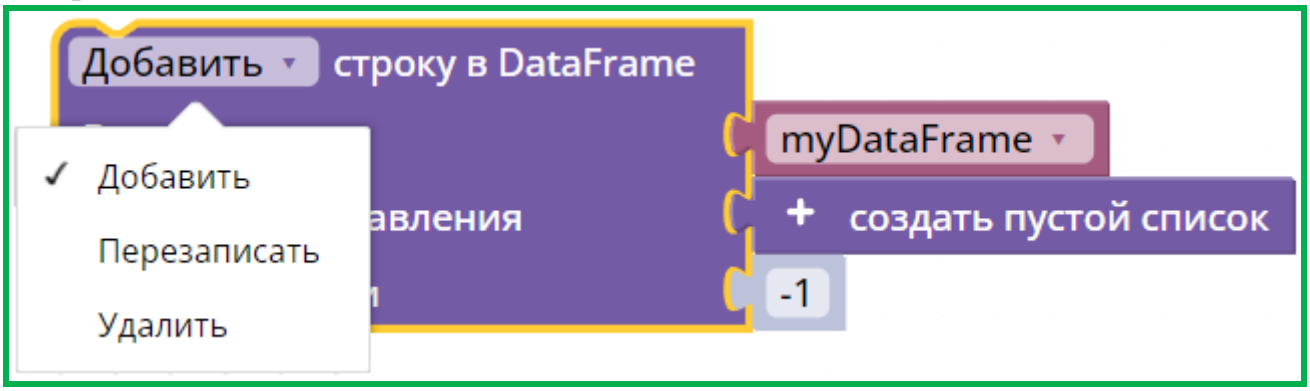


Блок «Добавить строку в *DataFrame*»

Блок «Добавить строку в *DataFrame*» – это основной инструмент для накопления данных в процессе автоматизации. Он позволяет добавить новую запись в существующую таблицу, сохраняя её структуру и типы данных.

Правильное использование этого блока делает процесс устойчивым и масштабируемым, особенно при работе с динамическими или повторяющимися данными.

Блок позволяет добавить, перезаписать и удалить строку в *DataFrame*. При этом необходимо указать сам *DataFrame*, строку для добавления/перезаписи и номер позиции.



1. Добавить строку в существующий *DataFrame*. Блок позволяет накапливать данные в цикле или по мере поступления информации.

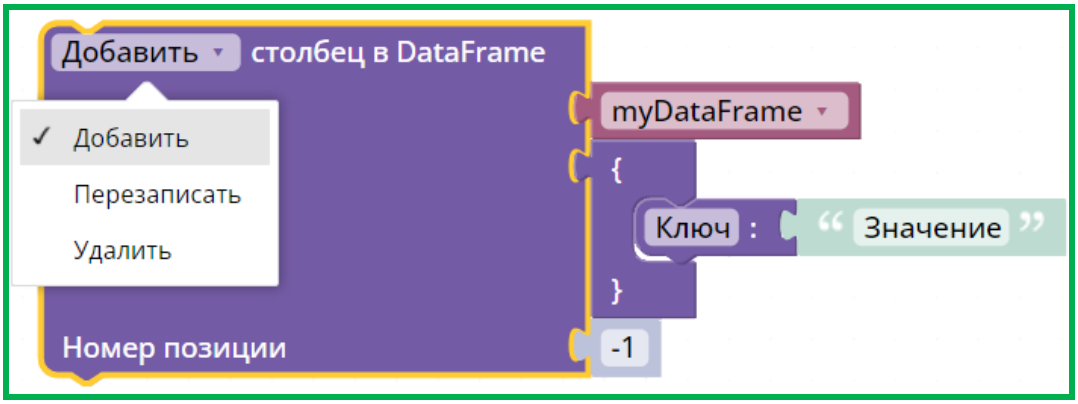
Добавление строки – это основной способ построчной сборки данных без разрушения структуры таблицы.

2. Перезаписать строку – позволяет изменить существующую запись в *DataFrame*, сохраняя структуру таблицы. Это важный инструмент для корректировки данных, обновления статусов и исправления ошибок в процессе автоматизации. Правильное использование блока помогает поддерживать актуальность и целостность таблицы, без необходимости создавать новую структуру или удалять старые строки вручную.

3. Удалить строку – позволяет удалять одну или несколько записей из *DataFrame*. Это важный инструмент для управления актуальностью данных, очистки таблицы и исключения ненужных или некорректных записей. Правильное использование блока помогает поддерживать чистоту и согласованность таблицы, избегая накопления лишних данных и ошибок в дальнейшем.

Блок «Добавить столбец в *DataFrame*»

Блок позволяет добавить, перезаписать и удалить столбец в *DataFrame*. При этом необходимо указать сам *DataFrame*, словарь для добавления/перезаписи и номер позиции.



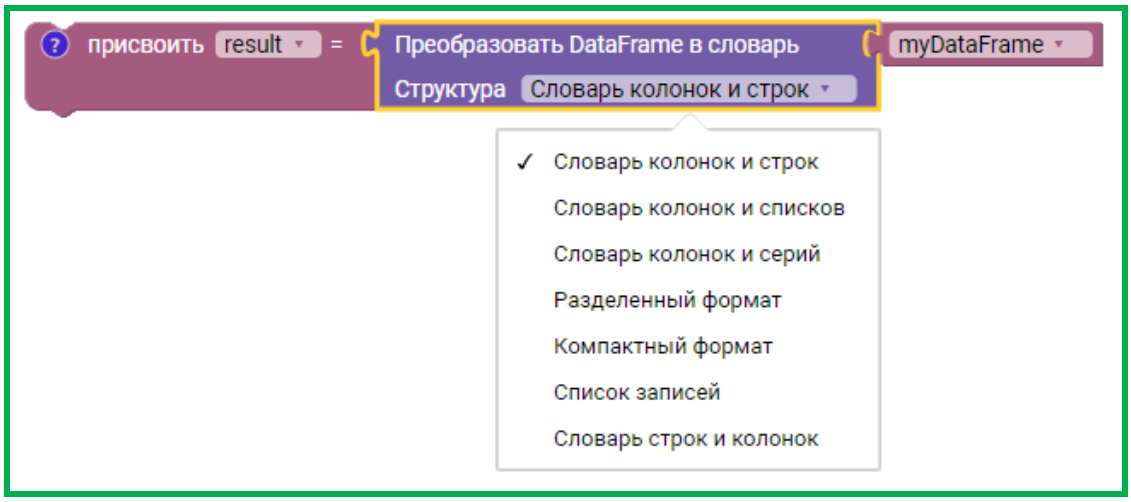
1. Блок «Добавить столбец» позволяет расширить существующую таблицу новым признаком или расчётным полем. Это ключевой инструмент для дополне-

ния информации, создания флагов обработки, расчётных полей и аналитики внутри *DataFrame*. Правильное использование блока помогает сделать таблицу полной, информативной и готовой для последующей обработки

2. Блок «Перезаписать столбец» позволяет изменить все значения одного столбца или отдельных ячеек в *DataFrame*. Это ключевой инструмент для обновления данных, корректировки расчётов и изменения статусовбез разрушения структуры таблицы. Использование этого блока делает *DataFrame* динамической и управляемой структурой, где можно оперативно менять данные на основе условий или результатов вычислений.

3. Блок «Удалить столбец» позволяет удалить один или несколько столбцов из *DataFrame*. Это важный инструмент для очистки таблицы, оптимизации структуры и подготовки данных к дальнейшей обработке. Правильное использование блока помогает поддерживать чистоту, актуальность и удобочитаемость таблицы-без разрушения её целостности.

Блок «Преобразовать *DataFrame* в словарь»

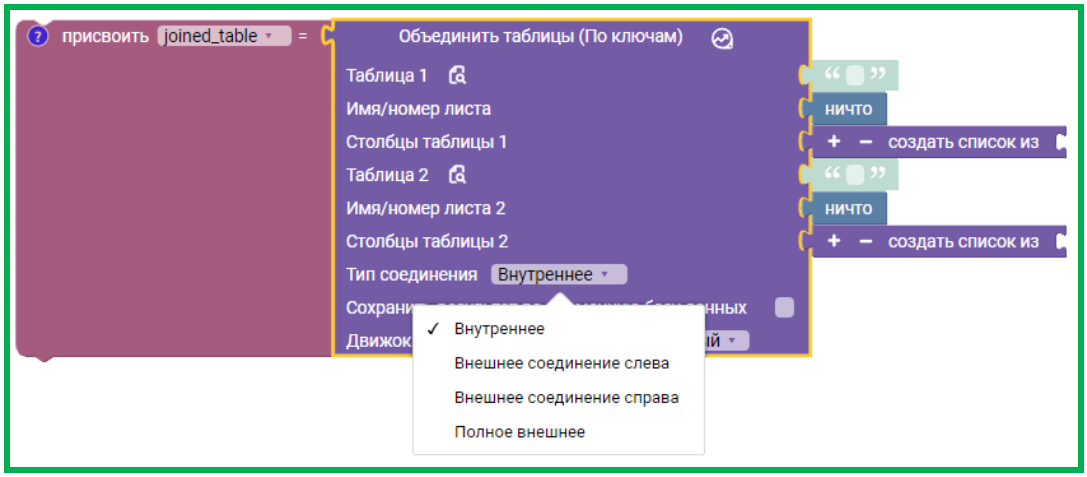


Блок позволяет преобразовать *DataFrame* в словарь со структурой:

- Словарь колонок и строк: Каждая колонка превращается в словарь, где ключи – это строки, а значения – данные из ячеек;
- Словарь колонок и списков: Каждая колонка превращается в список значений из всех строк;
- Словарь колонок и серий: Каждая колонка становится объектом Series (похож на список, но с дополнительными метками строк);
- Разделенный формат: Создает отдельные списки для строк, колонок и данных;
- Компактный формат: То же, что “Разделенный формат”, но дополнительно включает имена строк и колонок;
- Список записей: Каждый ряд превращается в словарь, где ключи – это имена колонок, а значения – данные;
- Словарь строк и колонок: Ключи – это строки, а значения – вложенные словари с колонками и их данными.

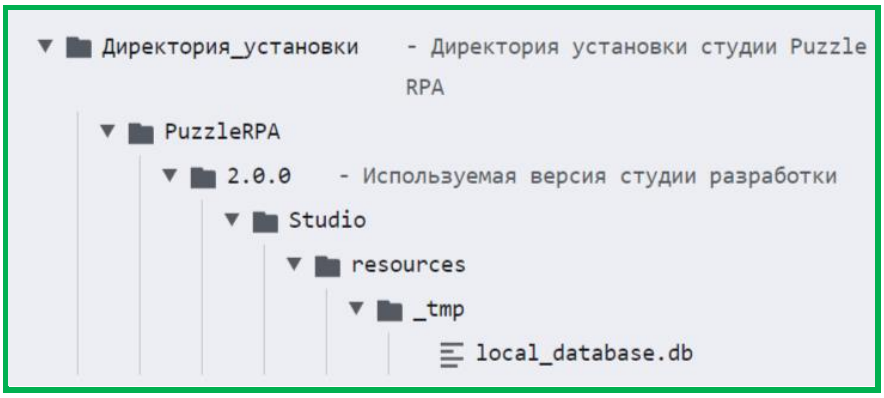
Этот блок особенно важен, когда требуется работать с отдельными записями или передавать данные вне *DataFrame*, сохраняя структуру и связь столбцов и строк.

Блок «Объединить таблицы (По Ключам)»



Блок позволяет объединить таблицы (По Ключам). Необходимо указать:

- Таблица 1 – Путь к файлу или *DataFrame* первого источника данных;
- Имя/номер листа – Имя или порядковый номер листа в таблице 1; Если в разъеме Имя/номер листа используется блок Ничто, будет использоваться первый лист книги Excel.
- Столбцы таблицы 1 – Ключи для первого источника;
- Таблица 2 – Путь к файлу или *DataFrame* второго источника данных;
- Имя/номер листа 2 – Имя или порядковый номер листа в таблице 2;
- Столбцы таблицы 2 – Ключи для второго источника;;
- Тип соединения – Определяет тип объединения:
  - ✓ **Внутреннее**: используются только те индексы, которые присутствуют в обоих таблицах;
  - ✓ **Внешнее соединение слева:** используются все индексы из первой таблицы, а индексы из второй таблицы, не совпадающие с первой, будут заполнены значениями NaN;
  - ✓ **Внешнее соединение справа:** используются все индексы из второй таблицы, а индексы из первой таблицы, не совпадающие с ней, будут заполнены значениями NaN;
  - ✓ **Полное внешнее:** используются все индексы из обоих таблицы. Если индекс отсутствует в одной из них, будет заполнено значением NaN.
- Сохранить результат во временную базу данных – чекбокс позволяет сохранить промежуточный результат в базу данных; временная база данных предназначена исключительно для разработки и отладки алгоритмов программных роботов и сохраняется в директорию:



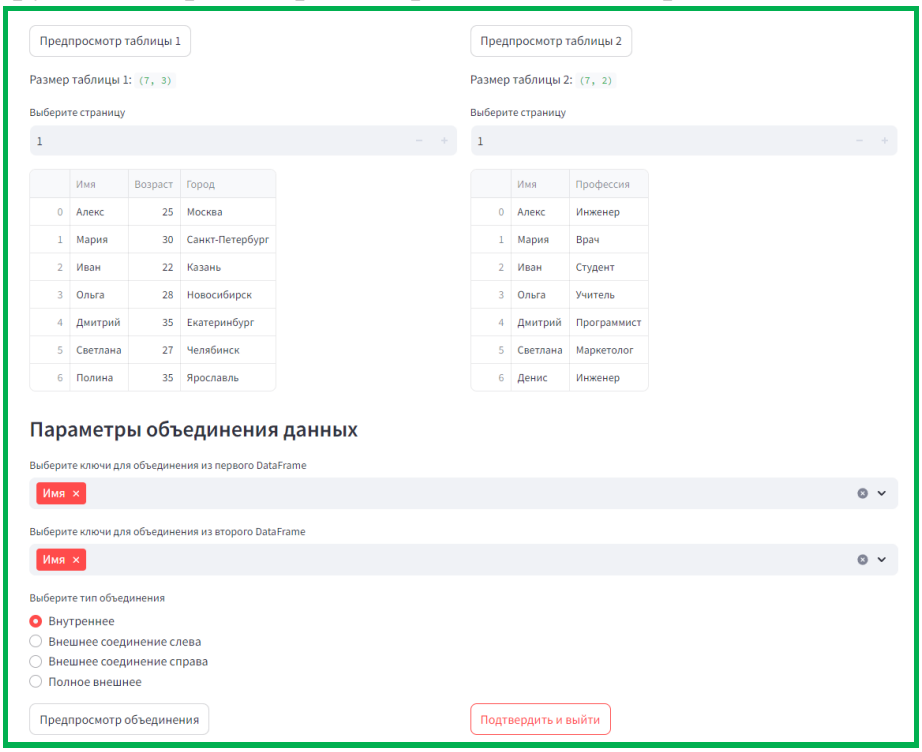
Для использования созданной базы данных необходимо указать путь к файлу local_database.db в блоках “Табличные данные” в поле “Таблица”.

- Движок обработки данных – Определяет способ использования ресурсов процессора при обработке таблиц.

Визуализация данных

Для визуальной работы с данными, кликните по кнопке: 

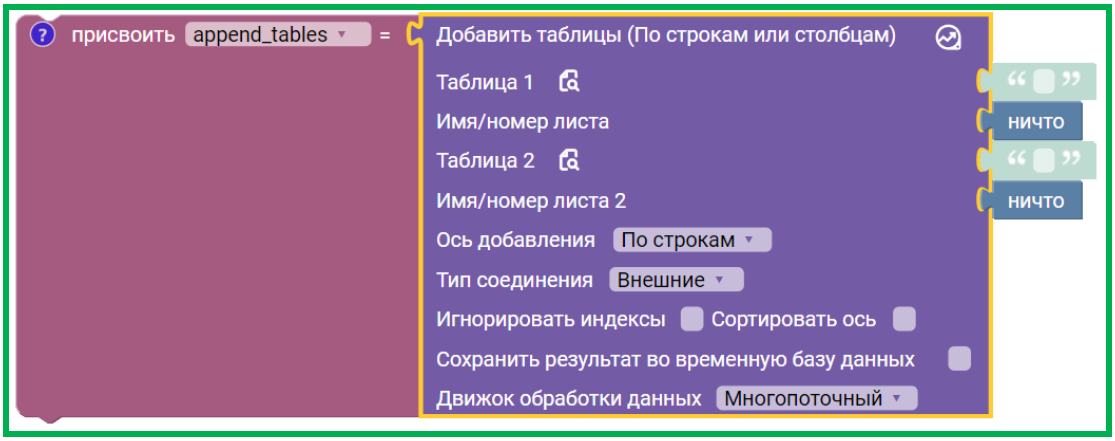
Форма визуализации данных – встроенный инструмент Puzzle RPA, который позволяет загружать и просматривать различные наборы данных.



Блок «Объединить таблицы» – это один из самых мощных инструментов для работы с табличными данными, аналогичный SQL JOIN, который используется для сопоставления и интеграции информации из разных источников.

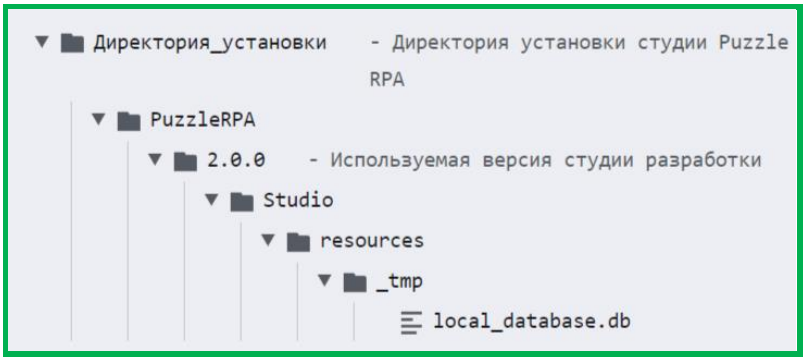
Правильное использование блока позволяет создавать консолидированные таблицы, готовые для анализа, отчётности и дальнейшей автоматизации.

Блок «Добавить таблицы (По Строкам или Столбцам)»



Блок позволяет добавить таблицы (По Строкам или Столбцам). Необходимо указать:

- Таблицу 1 – Путь к файлу или *DataFrame* первого источника данных;
- Имя/номер листа – Имя или порядковый номер листа в таблице 1; Если в разъеме Имя/номер листа используется блок Ничто, будет использоваться первый лист книги Excel.
- Таблицу 2 – Путь к файлу или *DataFrame* второго источника данных;
- Имя/номер листа 2 – Имя или порядковый номер листа в таблице 2;
- Ось добавления – Определяет направление объединения: **По строкам:** Новые строки будут добавлены ниже существующих; **По столбцам:** Новые столбцы будут добавлены справа от существующих.
- Тип соединения – определяет, как обрабатывать индексы, которые не совпадают: **Внешнее:** объединяет все индексы из всех объектов. Если индекс отсутствует в одном из объектов, в результате будет NaN (пустое значение); **Внутреннее:** объединяет только те индексы, которые присутствуют во всех объектах.
- Игнорировать индексы – Опция сбрасывает индексы в объединённом *DataFrame*;
- Сортировать ось – Опция сортирует оси, если требуется для объединения;
- Сохранить результат во временную базу данных – чекбокс позволяет сохранить промежуточный результат в базу данных; временная база данных предназначена исключительно для разработки и отладки алгоритмов программных роботов и сохраняется в директорию:



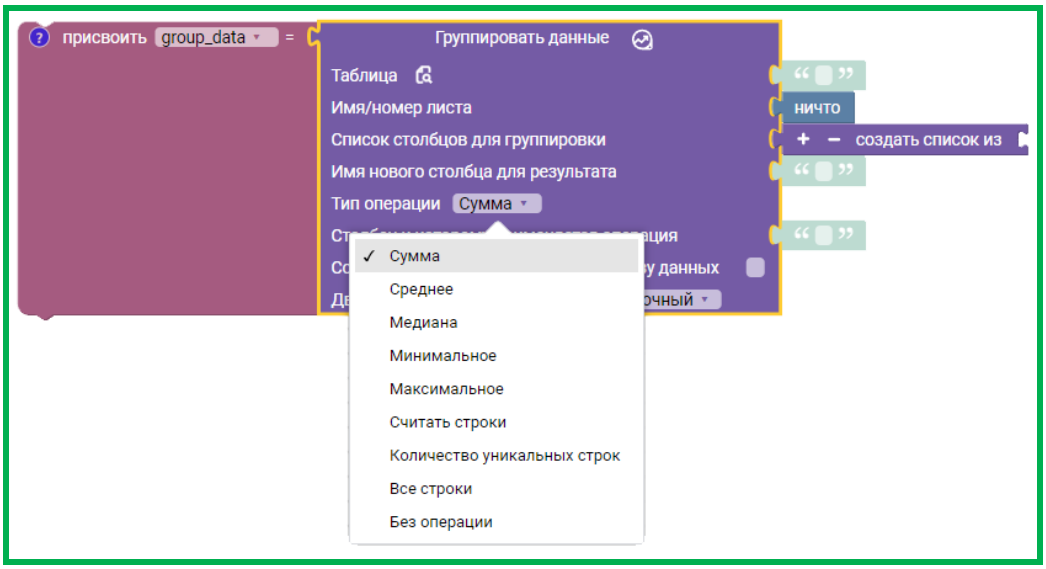
Для использования созданной базы данных необходимо указать путь к файлу local_database.db в блоках “Табличные данные” в поле “Таблица”.

- Движок обработки данных–Определяет способ использования ресурсов процессора при обработке таблиц.

  Для визуальной работы с данными, кликните по кнопке: 
  Добавление таблиц используется:
- для консолидации данных;
- для сборки промежуточных результатов;
- для построения единого реестра;
- для формирования отчёта.

  Это инструмент структурного расширения *DataFrame*.

Блок «Группировать табличные данные»



Блок позволяет объединить таблицы (По Ключам). Необходимо указать:

- Таблицу – Путь к файлу или *DataFrame* источника данных;
- Имя/номер листа – Имя или порядковый номер листа в таблице 1;
- Список столбцов для группировки – Столбец или список столбцов для группировки;
- Имя нового столбца для результата – Имя нового столбца для результата операции;
- Тип операции – Название операции:

✓ **Сумма:** Вычисляет общую сумму значений в выбранной колонке для каждой группы;

✓ **Среднее:** Рассчитывает среднее значение для выбранной колонки в каждой группе;

✓ **Медиана:** Определяет среднее значение ряда чисел, разделяя его на две равные части.

✓ **Минимальное:** Определяет наименьшее значение в выбранной колонке для каждой группы;

✓ **Максимальное:** Находит наибольшее значение в выбранной колонке для каждой группы;

✓ **Считать строки:** Подсчитывает общее количество строк в каждой группе;

- ✓ **Количество уникальных строк:** Подсчитывает количество уникальных значений в выбранной колонке для каждой группы;
- ✓ **Все строки:** Возвращает все строки данных, сгруппированных по выбранным критериям, без изменений;
- ✓ **Без операции:** Оставляет данные без каких-либо агрегирующих операций, показывая исходные значения.

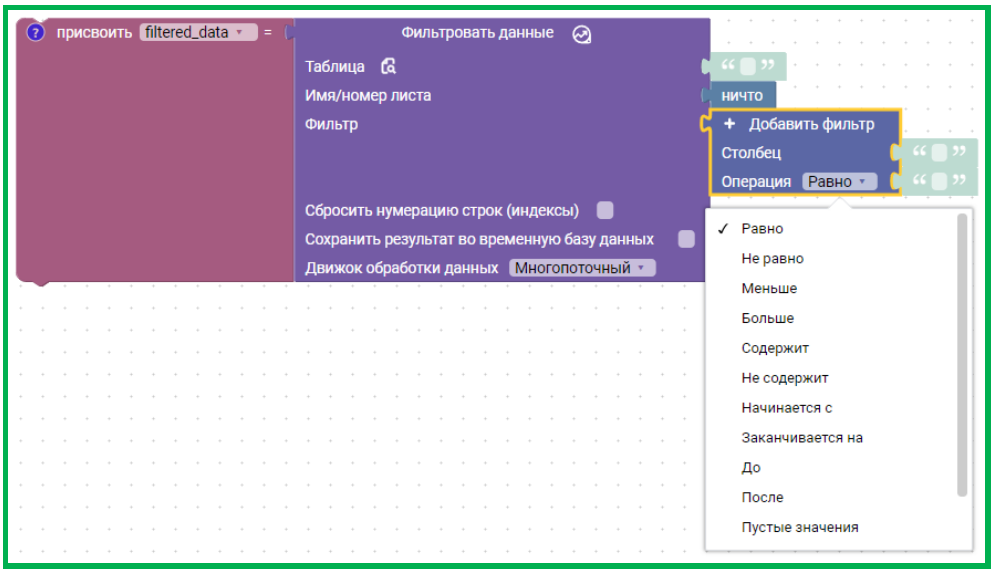
- Столбец к которому применяется операция–Имя столбца для применения операции;
- Сохранить результат во временную базу данных – Чекбокс позволяет сохранить промежуточный результат в базу данных;
- Движок обработки данных – Определяет способ использования ресурсов процессора при обработке таблиц.

Для визуальной работы с данными, кликните по кнопке: 

Блок «Группировать табличные данные» – это инструмент аналитики, который позволяет объединять строки *DataFrame* по одному или нескольким признакам и выполнять над ними агрегирующие операции (сумма, количество, среднее и т.д.).

Если фильтрация отвечает на вопрос «какие строки нам нужны?», то группировка отвечает на вопрос «какую сводную информацию мы можем из них получить?». Это уже уровень аналитического мышления в RPA

Блок «Фильтровать табличные данные»



Блок позволяет фильтровать табличные данные. Необходимо указать:

- Таблицу – Путь к файлу или *DataFrame* источника данных;
- Фильтр – Операции сравнения для фильтрации столбцов;
- Сбросить нумерацию строк (индексы) – Чекбокс позволяет восстановить стандартную нумерацию;
- Сохранить результат во временную базу данных – Чекбокс позволяет сохранить промежуточный результат в базу данных;
- Движок обработки данных – Определяет способ использования ресурсов процессора при обработке таблиц.

Для визуальной работы с данными, кликните по кнопке: 

Блок «Фильтровать табличные данные» позволяет отбирать строки *DataFrame* по заданным условиям.

Это один из самых часто используемых и самых важных блоков при работе с таблицами, потому что именно фильтрация превращает «сырой массив данных» в точечную, управляемую выборку для дальнейших действий.

Если объединение таблиц – это про расширение данных, то фильтрация – это про сужение и точность.

ПОДВЕДЕНИЕ ИТОГОВ ПРАКТИЧЕСКОЙ РАБОТЫ

В результате выполнения практической работы студенты научатся профессионально работать с табличными данными в Puzzle RPA, используя структуру *DataFrame* как основной инструмент обработки информации.

После завершения работы студенты научатся:

- Создавать структуру табличных данных. Научатся формировать *DataFrame* с нуля, задавать корректные названия столбцов и управлять типами данных.
- Управлять строками. Научатся: добавлять новые строки; перезаписывать существующие; удалять ненужные записи; контролировать целостность таблицы.
- Управлять столбцами. Научатся: добавлять новые столбцы; рассчитывать значения; перезаписывать данные; удалять временные и вспомогательные поля.
- Фильтровать данные. Научатся отбирать записи по условиям, строить сложные логические фильтры и формировать выборки для дальнейшей обработки.
- Группировать и анализировать информацию. Научатся создавать сводные таблицы, рассчитывать суммы, средние значения и количество записей по категориям.
- Объединять таблицы разными способами. Научатся различать: объединение по ключам (логическое сопоставление); добавление по строкам (вертикальное расширение); добавление по столбцам (горизонтальное расширение).
- Преобразовывать *DataFrame* в словарь. Научатся конвертировать таблицу в формат, удобный для передачи в формы, функции и внешние системы.

Дополнительно студенты научатся:

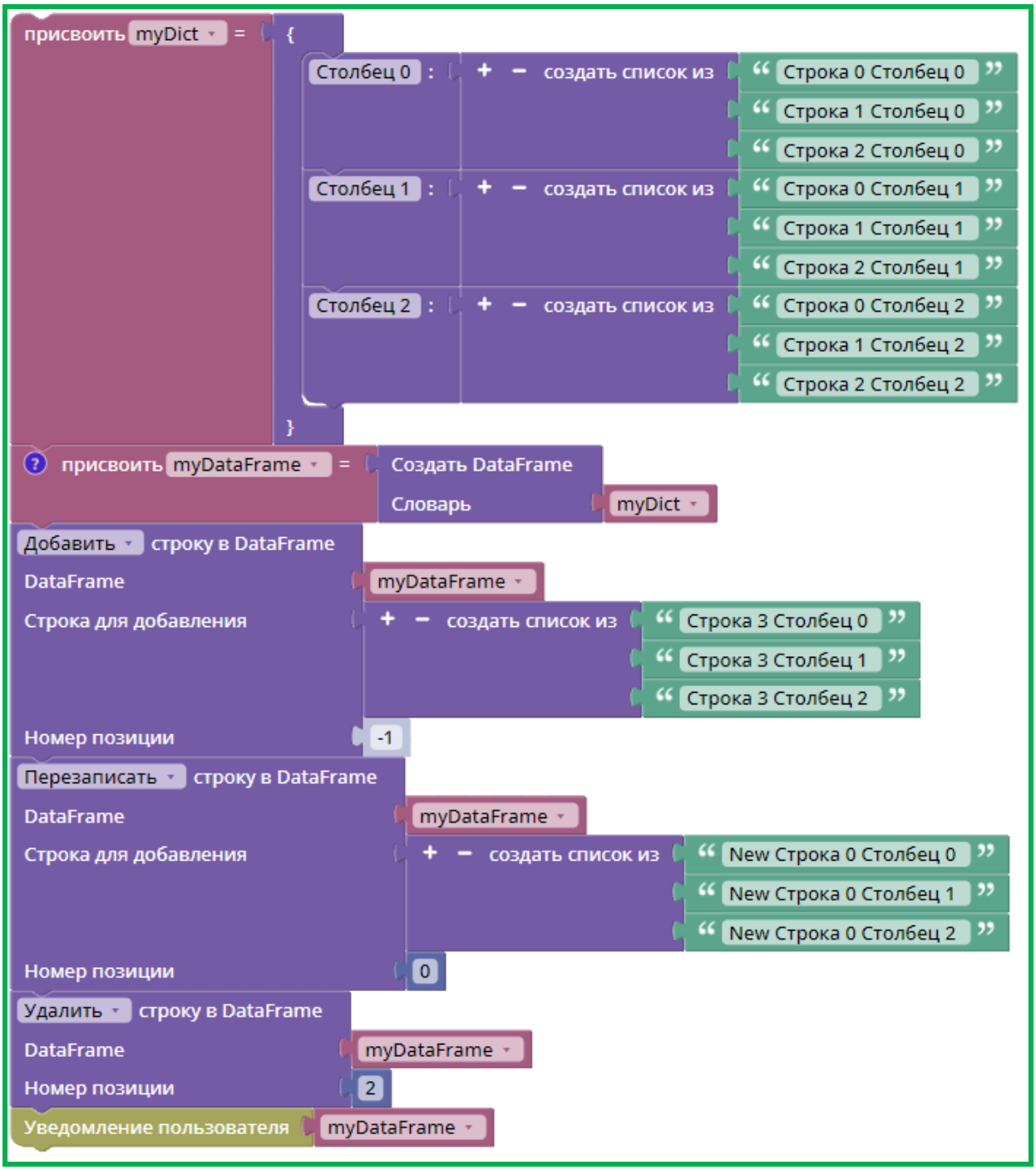
- выбирать оптимальный способ обработки данных;
- проектировать структуру RPA-процесса на основе табличной логики;
- минимизировать использование циклов за счёт табличных операций;
- предотвращать ошибки несоответствия типов данных;
- строить масштабируемую архитектуру автоматизации.

Работа с *DataFrame* станет не вспомогательным инструментом, а центральным механизмом автоматизации.

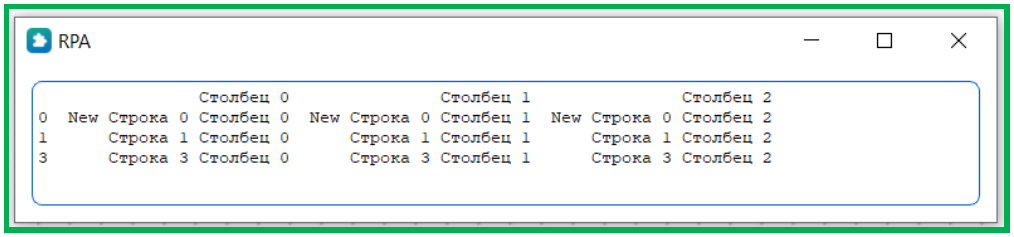
ПРИМЕРЫ ЗАДАЧ

Пример 1 – Изменение строк в *DataFrame*

В данном примере создается *DataFrame* на основе словаря, после чего в словарь добавляется новая строка, перезаписывается нулевая строка и удаляется вторая строка.

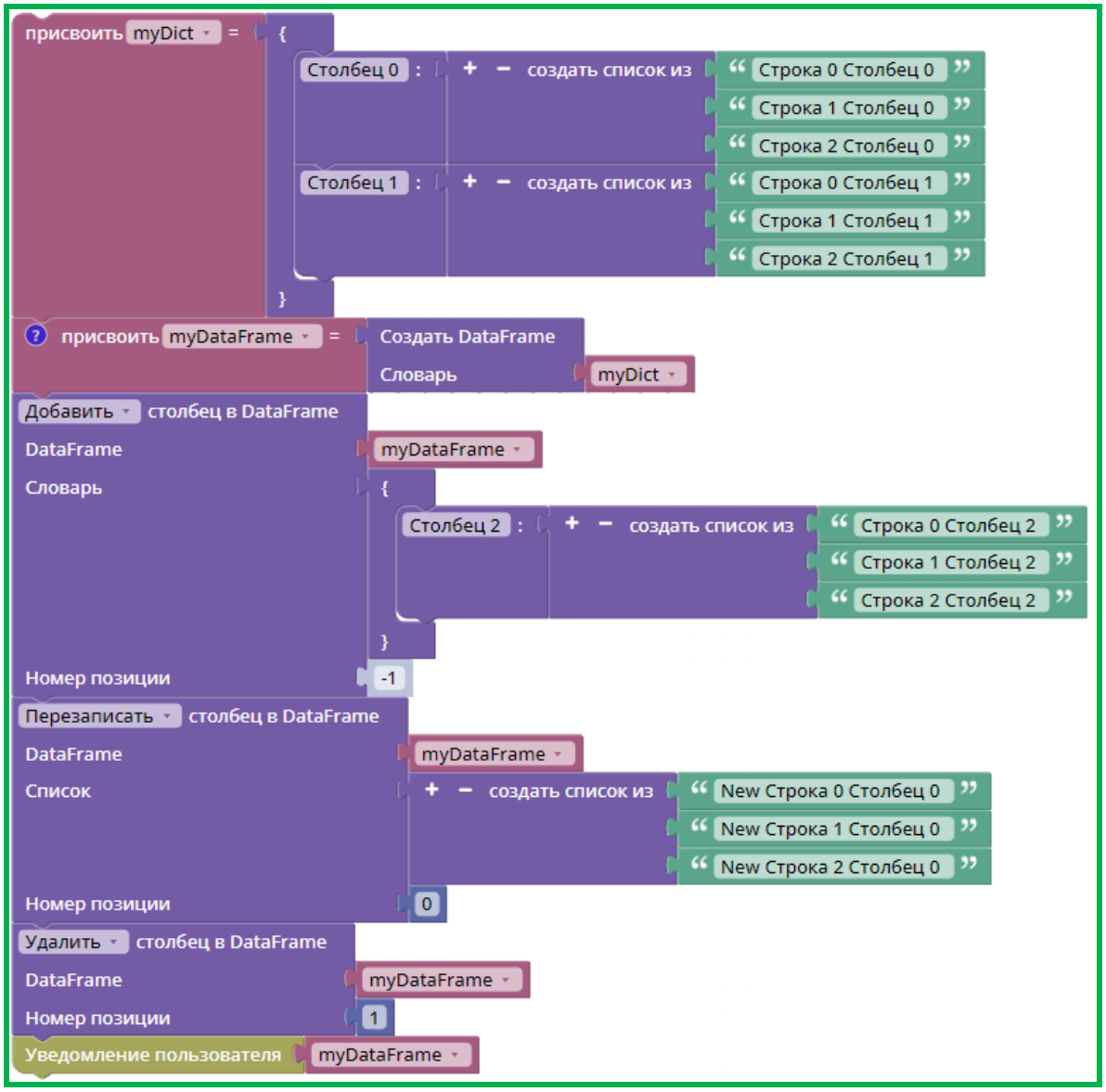


Вывод результата на экран:

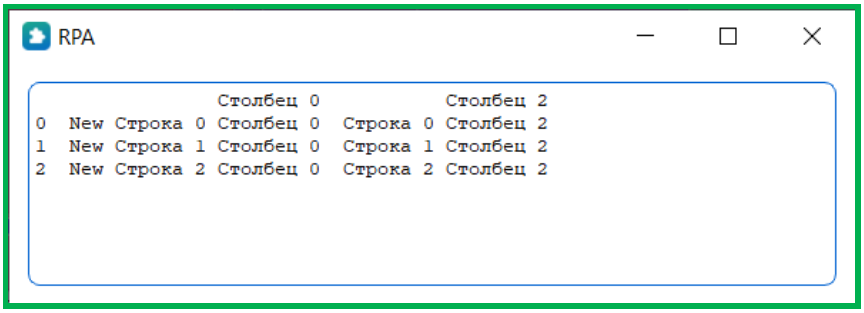


Пример 2 – Изменение столбцов в *DataFrame*

В данном примере на основе словаря создается *DataFrame*, после чего в *DataFrame* добавляется новый столбец, перезаписывается нулевой столбец и удаляется первый столбец.

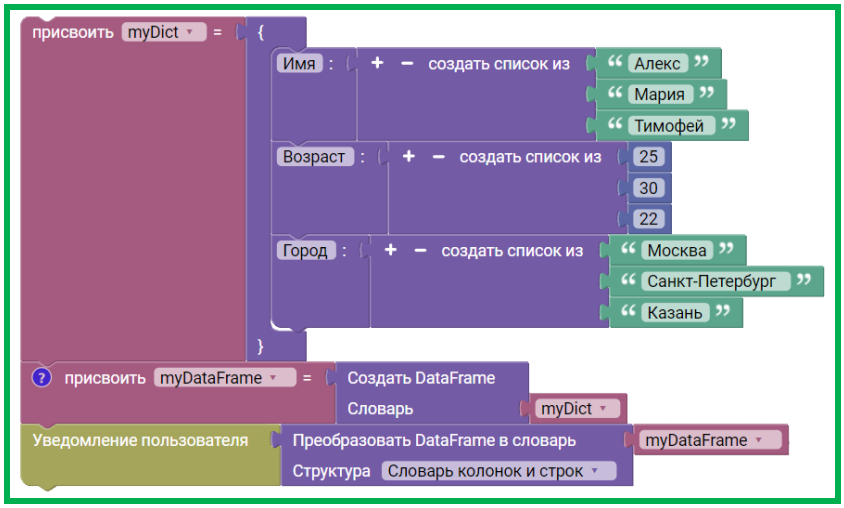


Вывод результата на экран:

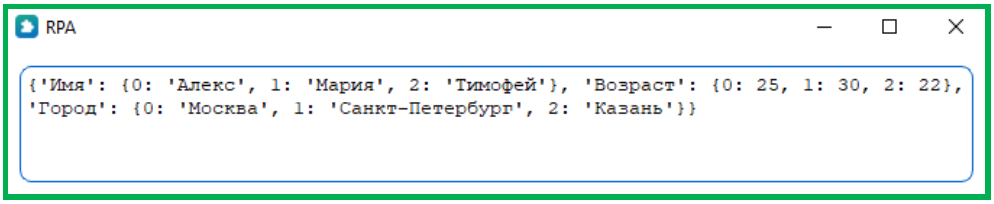


Пример 3 – Преобразование *DataFrame* в словарь

В данном примере блок преобразует *DataFrame* в словарь:



Вывод результата на экран:



Пример 4 – Объединение *DataFrame*

В данном примере блок объединяет две таблицы:

1. Выбирается первая таблица из файла-Excel со следующим содержанием:

	A	B	C
1	**Имя**	**Возраст**	**Город**
2	Алекс	25	Москва
3	Мария	30	Санкт-Петербург
4	Иван	22	Казань
5	Ольга	28	Новосибирск
6	Дмитрий	35	Екатеринбург
7	Светлана	27	Челябинск
8	Полина	35	Ярославль
9			

2. Выбирается вторая таблица из файла-Excel со следующим содержанием:

	A	B
1	**Имя**	**Профессия**
2	Алекс	Инженер
3	Мария	Врач
4	Иван	Студент
5	Ольга	Учитель
6	Дмитрий	Программист
7	Светлана	Маркетолог
8	Денис	Инженер
9		
10		

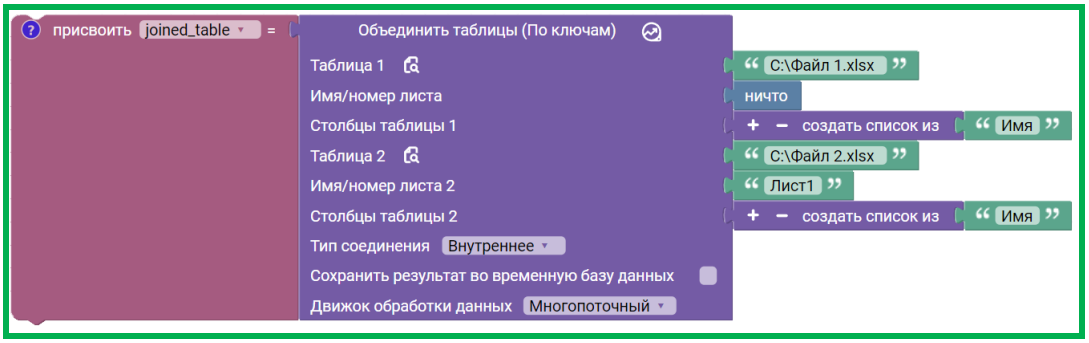
В качестве ключа для объединения, указывается общий столбец `Имя` для обоих таблиц. В каждой таблице присутствуют уникальные значения для данного ключа.

Результаты для каждого типа соединения:

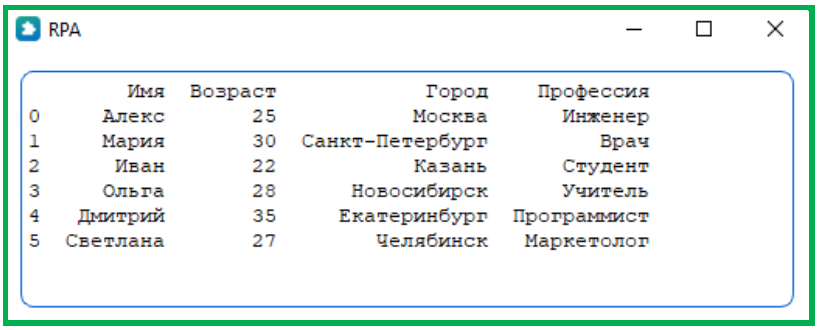
- Внутреннее:

	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	Инженер
1	Мария	30	Санкт-Петербург	Врач
2	Иван	22	Казань	Студент
3	Ольга	28	Новосибирск	Учитель
4	Дмитрий	35	Екатеринбург	Программист
5	Светлана	27	Челябинск	Маркетолог

Результат выводится на экран.



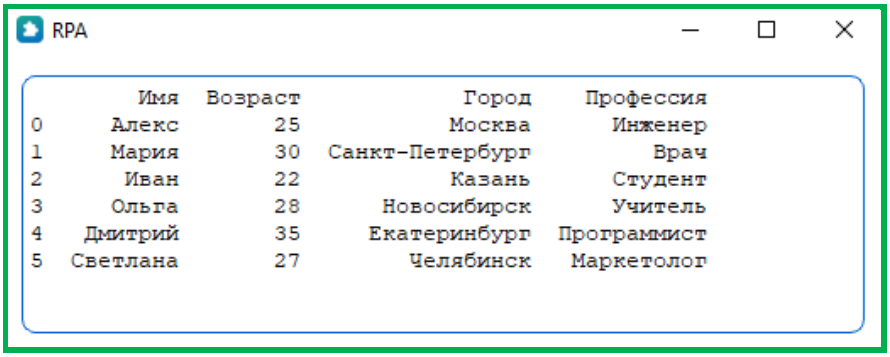
Вывод результата на экран:



- Внешнее соединение слева:

	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	Инженер
1	Мария	30	Санкт-Петербург	Врач
2	Иван	22	Казань	Студент
3	Ольга	28	Новосибирск	Учитель
4	Дмитрий	35	Екатеринбург	Программист
5	Светлана	27	Челябинск	Маркетолог
6	Полина	35	Ярославль	None

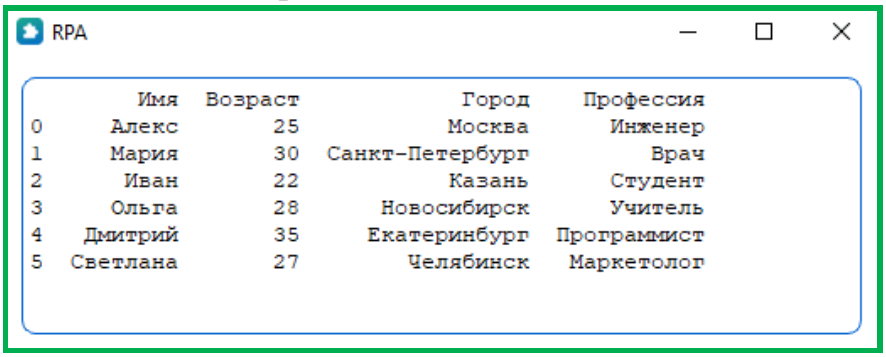
Результат выводится на экран.



- Внешнее соединение справа:

	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	Инженер
1	Мария	30	Санкт-Петербург	Врач
2	Иван	22	Казань	Студент
3	Ольга	28	Новосибирск	Учитель
4	Дмитрий	35	Екатеринбург	Программист
5	Светлана	27	Челябинск	Маркетолог
6	Денис	None	None	Инженер

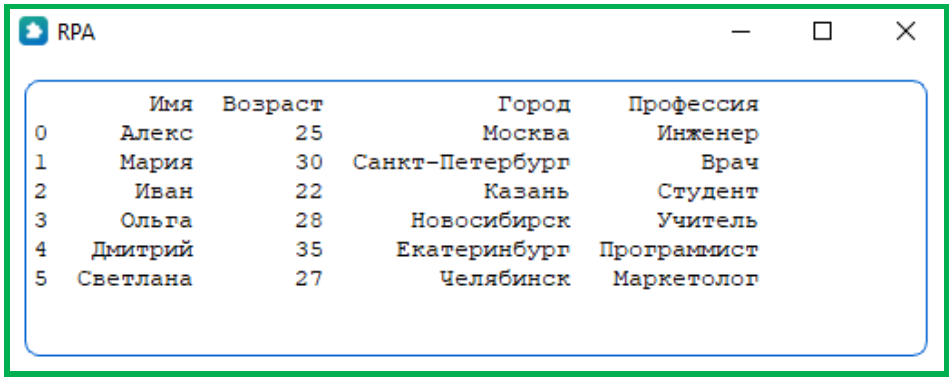
Результат выводится на экран.



- Полное внешнее:

	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	Инженер
1	Денис	None	None	Инженер
2	Дмитрий	35	Екатеринбург	Программист
3	Иван	22	Казань	Студент
4	Мария	30	Санкт-Петербург	Врач
5	Ольга	28	Новосибирск	Учитель
6	Полина	35	Ярославль	None
7	Светлана	27	Челябинск	Маркетолог

Результат выводится на экран.



Пример 5 –Добавить *DataFrame*

В данном примере блок добавляет таблицу к исходной:

- Выбирается первая таблица из файла-Excel со следующим содержанием:

	A	B	C
1	**Имя**	**Возраст**	**Город**
2	Алекс	25	Москва
3	Мария	30	Санкт-Петербург
4	Иван	22	Казань
5	Ольга	28	Новосибирск
6	Дмитрий	35	Екатеринбург
7	Светлана	27	Челябинск
8	Полина	35	Ярославль
9	Глеб	31	Воронеж
10	Юлия	43	Сочи

- Выбирается вторая таблица из файла-Excel со следующим содержанием:

	A	B	C	D
1	**Имя**	**Возраст**	**Город**	**Профессия**
2	Алекс	25	Москва	Инженер
3	Мария	30	Санкт-Петербург	Врач
4	Иван	22	Казань	Студент
5	Ольга	28	Новосибирск	Учитель
6	Дмитрий	35	Екатеринбург	Программист
7	Светлана	27	Челябинск	Маркетолог
8	Полина	35	Ярославль	Инженер
9	Глеб	31	Воронеж	Студент

Выбирается Ось добавления:

- По строкам

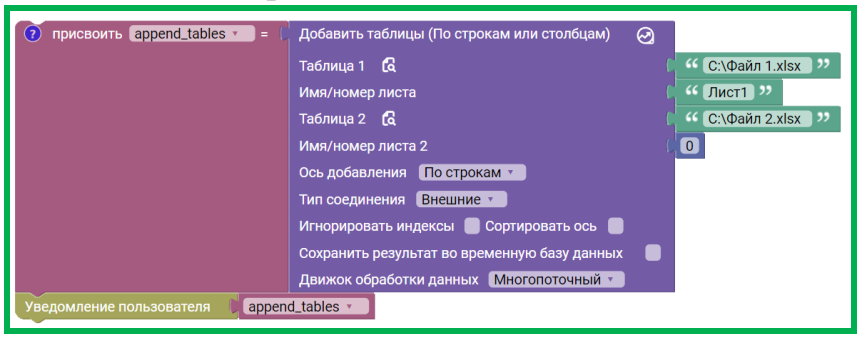
	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	None
1	Мария	30	Санкт-Петербург	None
2	Иван	22	Казань	None
3	Ольга	28	Новосибирск	None
4	Дмитрий	35	Екатеринбург	None
5	Светлана	27	Челябинск	None
6	Полина	35	Ярославль	None
7	Глеб	31	Воронеж	None
8	Юлия	43	Сочи	None
9	Алекс	25	Москва	Инженер

Выбирается Тип соединения:

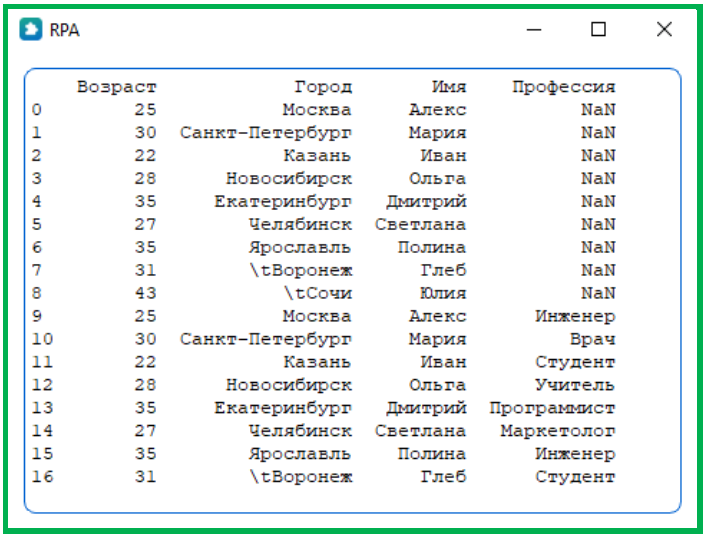
- Внешнее

	Имя	Возраст	Город	Профессия
0	Алекс	25	Москва	None
1	Мария	30	Санкт-Петербург	None
2	Иван	22	Казань	None
3	Ольга	28	Новосибирск	None
4	Дмитрий	35	Екатеринбург	None
5	Светлана	27	Челябинск	None
6	Полина	35	Ярославль	None
7	Глеб	31	Воронеж	None
8	Юлия	43	Сочи	None
9	Алекс	25	Москва	Инженер

Выбираются опции: “Игнорировать индексы” и “Сортировать ось”.
Результат выводится на экран.



Вывод результата на экран:



Пример 6 –Группировать *DataFrame*

В данном примере блок группирует данные:

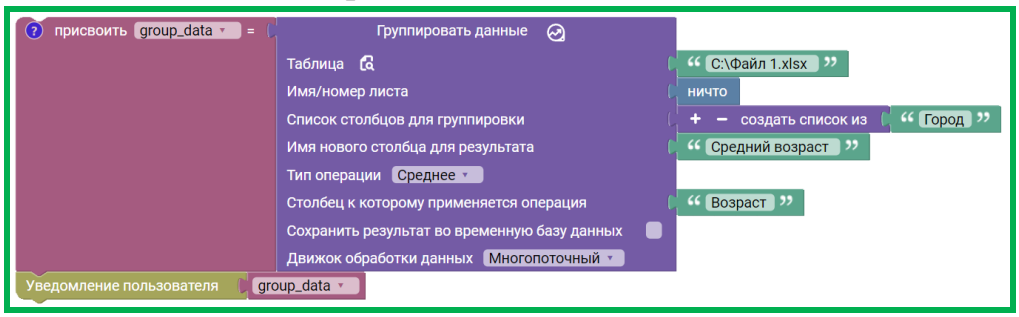
- Выбирается таблица из файла-Excel со следующим содержанием:

	A	B	C	D
1	**Имя**	**Возраст**	**Город**	**Профессия**
2	Алексей	25	Москва	Инженер
3	Мария	30	Санкт-Петербург	Врач
4	Иван	22	Москва	Студент
5	Ольга	35	Санкт-Петербург	Учитель
6	Дмитрий	28	Екатеринбург	Программист
7	Анна	40	Москва	Менеджер
8	Сергей	30	Екатеринбург	Архитектор
9	Елена	27	Санкт-Петербург	Дизайнер
10				

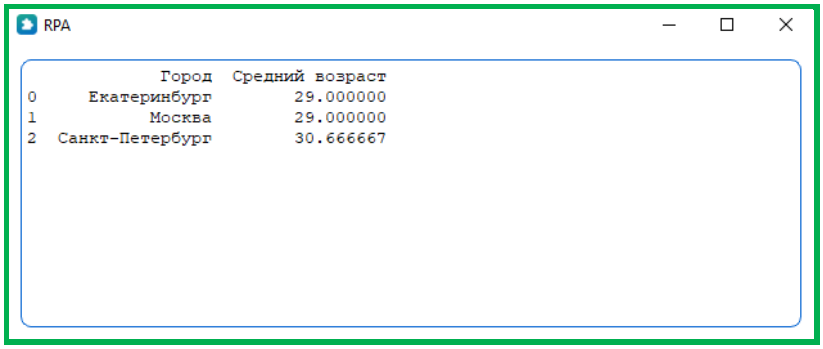
- В качестве Столбца для группировки выбирается “Город”;
- Указывается Имя нового столбца “Средний возраст” для результата;
- В качестве Столбца к которому применяется операция выбирается “Возраст”;
- В качестве Типа операции выбирается “Среднее”;

	Город	new_column
0	Екатеринбург	29
1	Москва	29
2	Санкт-Петербург	30.6667

- Результат выводится на экран.



Вывод результата на экран:



Пример 7 – Фильтровать *DataFrame*

В данном примере блок фильтрует данные:

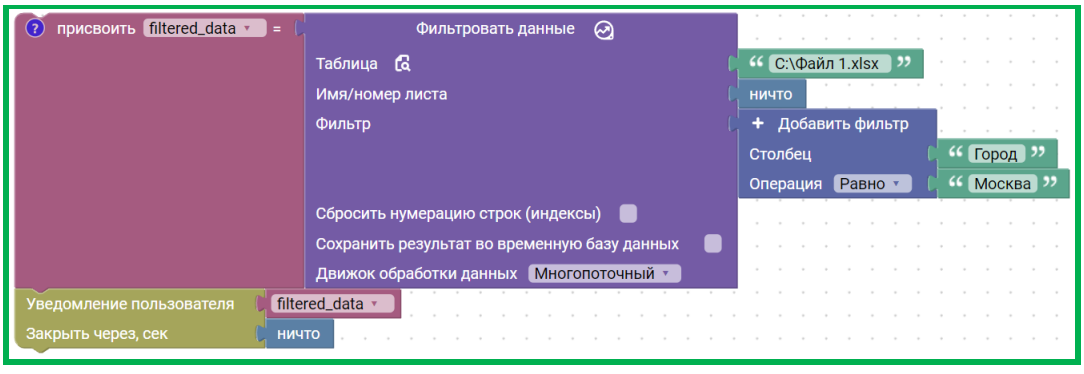
- Выбирается таблица из файла-Excel со следующим содержанием:

	A	B	C	D
1	**Имя**	**Возраст**	**Город**	**Профессия**
2	Алексей	25	Москва	Инженер
3	Мария	30	Санкт-Петербург	Врач
4	Иван	22	Москва	Студент
5	Ольга	35	Санкт-Петербург	Учитель
6	Дмитрий	28	Екатеринбург	Программист
7	Анна	40	Москва	Менеджер
8	Сергей	30	Екатеринбург	Архитектор
9	Елена	27	Санкт-Петербург	Дизайнер
10				

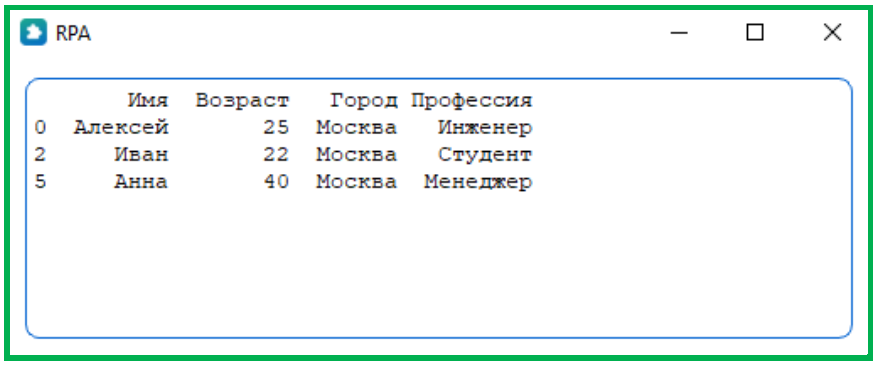
- В качестве Столбца для фильтрации выбирается “Город”;
- Указывается Тип операции “Равно”;
- В качестве Значения указывается “Москва”;

	Имя	Возраст	Город	Профессия
0	Алексей	25	Москва	Инженер
2	Иван	22	Москва	Студент
5	Анна	40	Москва	Менеджер

- Результат выводится на экран.



Вывод результата на экран:



КЕЙС. АНАЛИТИКА ЗАКАЗОВ И КОНТРОЛЬ ПРОСРОЧЕННЫХ ПЛАТЕЖЕЙ

Компания EcoDelivery занимается доставкой экологичных товаров по городам России. Каждый день обрабатываются десятки заказов: часть оплачена, часть – в обработке, а часть – отменена.

Руководство хочет видеть, какие заказы просрочены, где платежи задерживаются, а также формировать отчёт по активности клиентов.

Ваша задача как RPA-разработчика – автоматизировать работу с заказами через DataFrame в Puzzle RPA, с возможностью фильтровать данные, добавлять аналитические столбцы, сохранять результаты и уведомлять пользователя.

Задача 1

1. Создать исходный файл orders.xlsx с 10 записями: orders.xlsx
2. Считать исходный файл в объект в *DataFrame* `orders_df`
3. Добавить столбец `is_late` – флаг просрочки:

```
orders_df["is_late"] = orders_df["order_date"] < (today - 3)
```

4. Отфильтровать заказы по городу "Москва":
5. Сохранить результат в Excel и уведомить пользователя:

```
("Фильтрованные заказы по Москве успешно сохранены в moscow_orders.xlsx!")
```

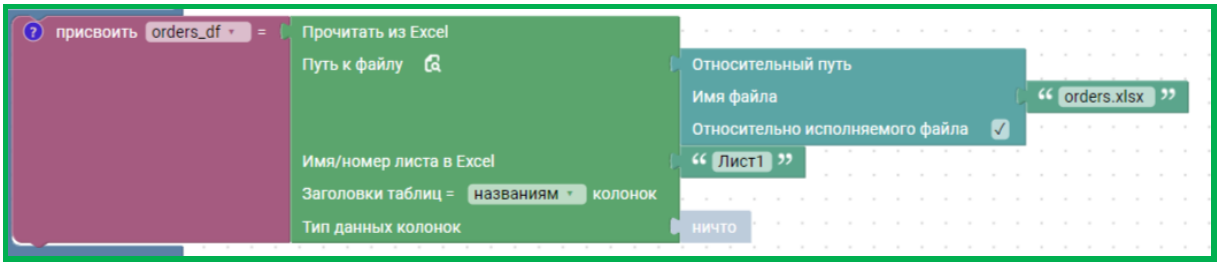
Шаг 1: Чтение файла заказов

Название блока: Прочитать из Excel

Переменная: `orders_df`

Настройки:

- Путь к файлу: относительный путь к файлу orders.xlsx.
- Лист: Лист1



Что делает: Читает данные из Excel-файла и сохраняет в переменную `orders_df` типа `DataFrame` (таблица с колонками).

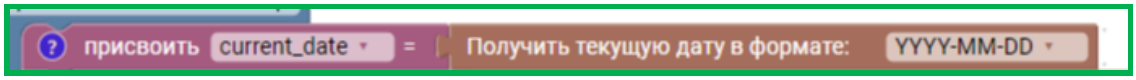
Шаг 2: Получение текущей даты

Название блока: Получить текущую дату

Переменная: `current_date`

Настройки:

- Формат: год-месяц-день (например, 2026-03-12).



Что делает: Возвращает текущую системную дату в заданном формате.

Формат на выходе: строка (`str`)

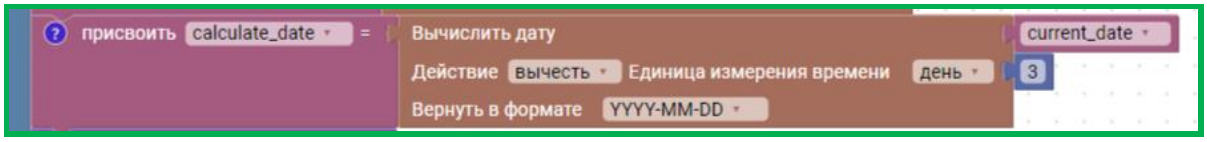
Шаг 3: Вычисление даты (минус 3 дня)

Название блока: Вычислить дату

Переменная: `calculate_date`

Настройки:

- Исходная дата: `current_date`
- Операция: `subtract` (вычитание)
- Формат вывода: год-месяц-день (например, 2026-03-12).
- Параметры: `days = 3`



Что делает: Отнимает от текущей даты 3 дня. Используется для определения порога просрочки.

Формат на выходе: строка (`str`)

Шаг 5: Создание пустого списка

Название блока: Присвоить значение переменной

Переменная: `is_late_list`

Значение: `[]` (пустой список)



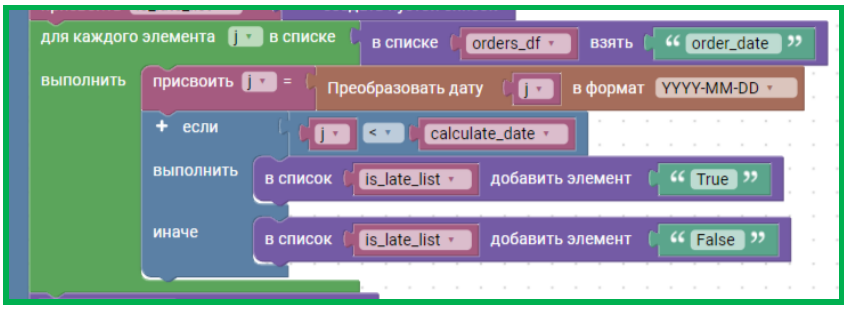
Что делает: Инициализирует пустой список для хранения флагов просрочки.

Шаг 6: Цикл по элементам столбца

Название блока: Цикл для каждого элемента

Переменная цикла: `j`

Источник: `orders_df['order_date']` – столбец с датами заказов.



Что делает: Перебирает каждую дату заказа.

Шаг 7: Преобразование даты

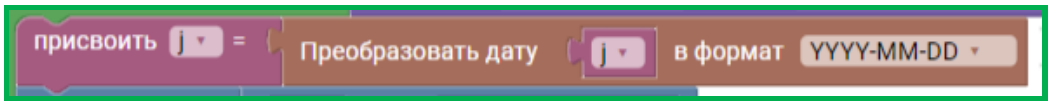
Название блока: Преобразовать дату

Переменная: j (перезаписывается)

Исходная дата: значение из цикла

Входной формат: определяется автоматически (из Excel) возвращается со строкой время, поэтому нам необходимо преобразование.

Выходной формат: год-месяц-день (например, 2026-03-12).



Что делает: Приводит дату к единому строковому формату для сравнения.

Шаг 8: Условие (если дата < пороговой)

Название блока: Если – выполнить

Условие: *j* < *calculate_date*



Что делает: Проверяет, является ли заказ просроченным (дата заказа раньше расчётной даты).

В ветвления условий добавляем результат.

Название блока: Добавить элемент в список

Список: `is_late_list`

Значение: `'True'` или `'False'`

Добавляет строковое значение в список в зависимости от выполнения условия.

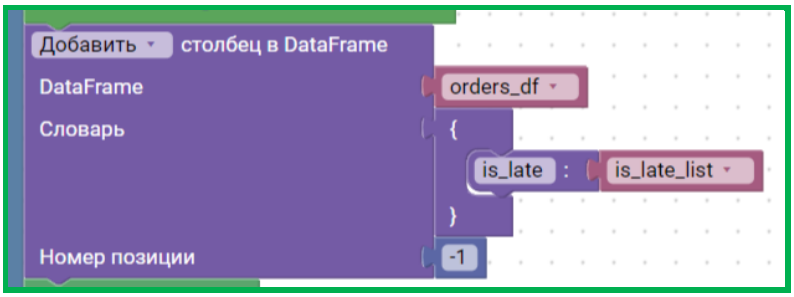
Шаг 9: Добавление столбца в DataFrame

Название блока: Добавить столбец в dataframe

Переменная: `orders_df` (изменяется исходная таблица)

Настройки:

- DataFrame: `orders_df`
- Данные: ключ `is_late` значение `is_late_list`
- Позиция: `-1` (в конец).



Что делает: Заполняет ранее добавленный столбец `is_late` значениями из списка

Формат на выходе: *DataFrame* с заполненным столбцом.

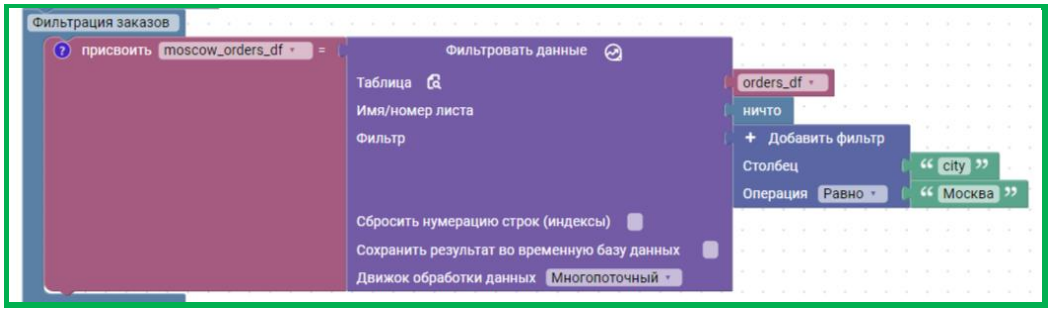
Шаг 10: Фильтрация DataFrame (заказы Москвы)

Название блока: Фильтровать табличные данные

Переменная: `moscow_orders_df`

Настройки:

- Источник: `orders_df`
- Условие: `city = 'Москва'`



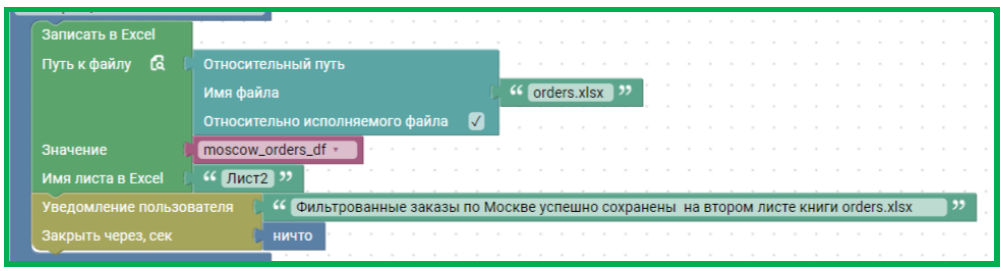
Что делает: Оставляет только строки, где город – Москва.

Шаг 11: Запись в Excel (Москва)

Название блока: Записать в Excel

Настройки:

- Путь: `orders.xlsx`
- Данные: `moscow_orders_df`
- Лист: `Лист2`



Что делает: Сохраняет отфильтрованные данные на второй лист файла orders.xlsx.

Шаг 12: Уведомление пользователя

Название блока: Уведомление пользователя

Текст: "Фильтрованные заказы по Москве успешно сохранены ..."

Что делает: Выводит всплывающее уведомление в интерфейсе Puzzle RPA.

Задача 2

После фильтрации по Москве руководство хочет видеть задолженности и просрочки платежей.

Существует второй файл *payments.xlsx*, в котором указаны фактические платежи клиентов.

Нужно объединить данные и выделить просроченные и частично оплаченные заказы, а результат сохранить и показать пользователю.

1. Создать файл payments.xlsx с 8 записями: *payments.xlsx*
2. Считать исходный файл в объект в *DataFrame* `payments_df`
3. Объединить `moscow_orders_df` и `payments_df` по `order_id`.
4. Добавить расчет `remaining_amount`:

```
combined_df["remaining_amount"] = combined_df["amount"] - combined_df["payment_amount"]
```

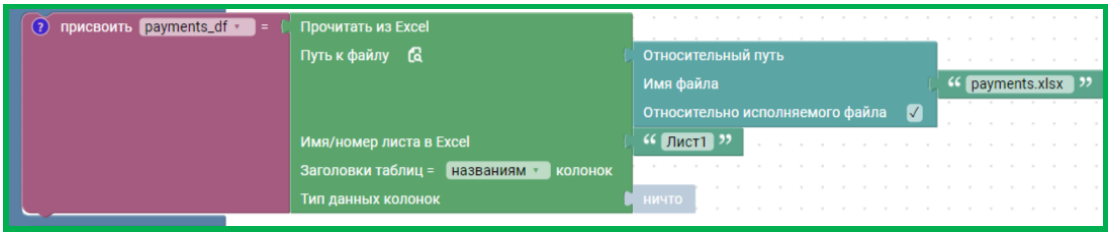
5. Выбрать просроченные заказы с задолженностью > `0`:
6. Сохранить результат в Excel и уведомить пользователя:

Шаг 1. Чтение файла платежей

Аналогично шагу 1 в первом задании, но:

Файл: `payments.xlsx`

Переменная: `payments_df`



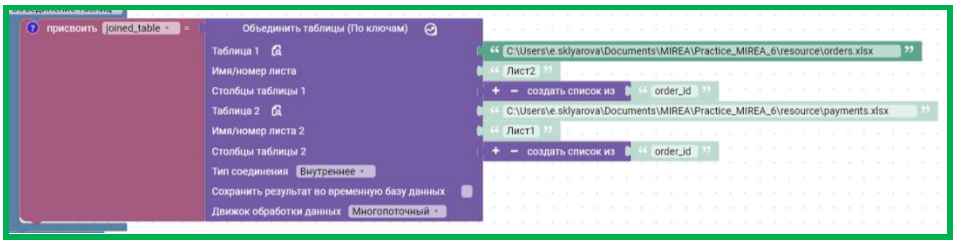
Шаг 2: Объединение таблиц по ключу

Название блока: Объединить Таблицы (По Ключам)

Переменная: `joined_table`

Настройки:

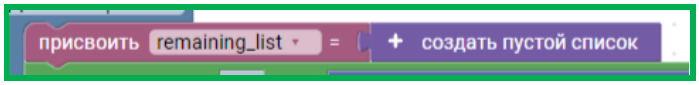
- Таблица 1: `orders.xlsx`, лист `Лист2`
- Ключ слева: `order_id`
- Таблица 2: `payments.xlsx`, лист `Лист1`
- Ключ справа: `order_id`
- Тип объединения: `inner` (только совпадающие)



Что делает: Объединяет заказы и платежи по `order_id`.

Шаг 3: Подготовка списка остатков

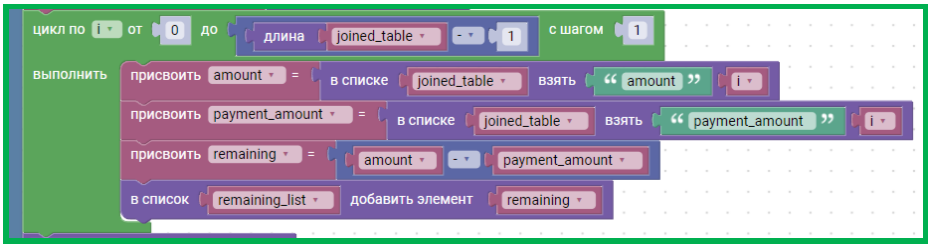
Переменная: `remaining_list = []`



Шаг 4: Цикл по индексам таблицы

Название: Цикл по (от до с шагом)

Диапазон: от `0` до `len(joined_table)-1`, шаг `1`



Что делает: Перебирает строки объединённой таблицы по индексу.

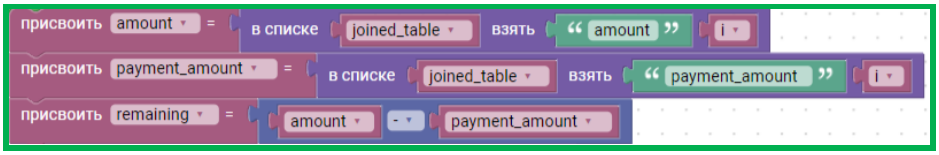
Шаг 5: Чтение полей и расчёт

`amount = joined_table['amount'][i]` – сумма заказа.

`payment_amount = joined_table['payment_amount'][i]` – оплачено.

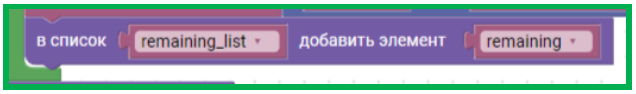
`remaining = amount - payment_amount` – остаток.

Формат данных: числа (`int/float`)



Шаг 6: Добавление остатка в список

Список: `remaining_list.append(remaining)`

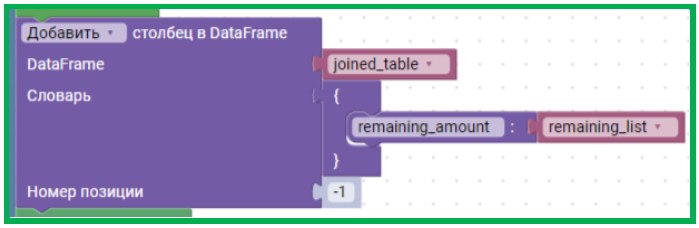


Шаг 7: Добавление столбца с остатками

Название: Добавить столбец в *dataframe*

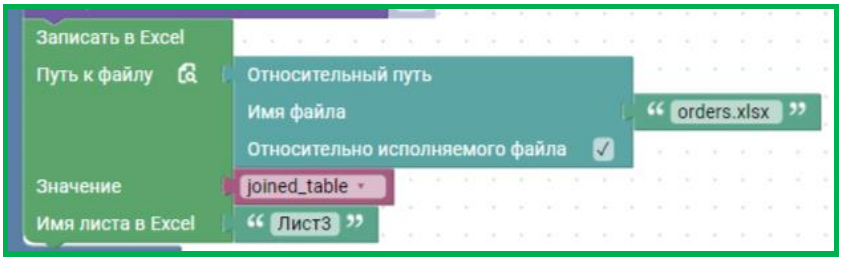
Столбец: `{'remaining_amount': remaining_list}`

Позиция: `-1`



Шаг 8: Запись объединённой таблицы

Лист: `Лист3` в `orders.xlsx`



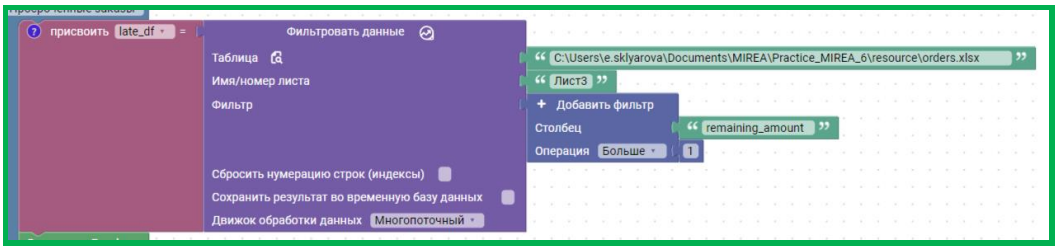
Шаг 9: Фильтрация просроченных заказов

Условие: `remaining_amount > 1`

В блоке фильтрации используется условие `remaining_amount > 1`, а не `> 0`, чтобы исключить из выборки технические погрешности вычислений и микроскопические остатки, которые не имеют практического значения. Значение 1 выступает как порог существенности.

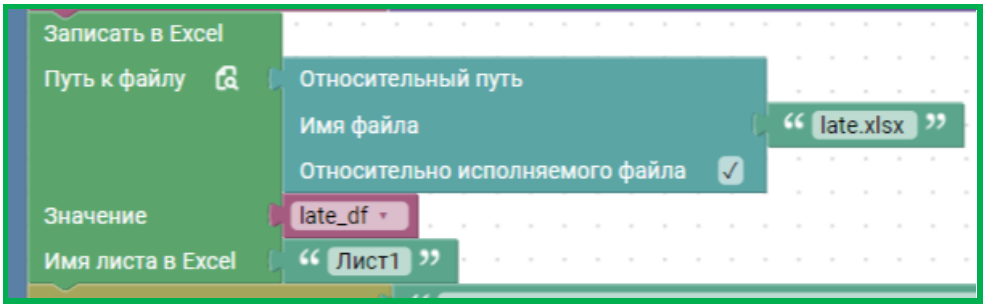
Источник: `orders.xlsx`, лист `Лист3`

Переменная: `late_df`



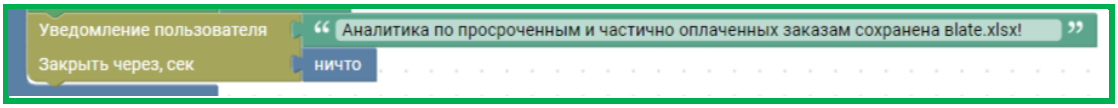
Шаг 10: Запись в новый файл

Файл: `late.xlsx`, лист `Лист1`



Блок 11: Уведомление

Текст: "*Аналитика по просроченным ... сохранена в late.xlsx!*"



В ходе выполнения данной работы вы познакомились с основными возможностями платформы Puzzle RPA для автоматизации обработки табличных данных. Разработанный робот демонстрирует типовой сценарий работы с данными в корпоративной среде: чтение исходных файлов, преобразование данных, аналитические расчёты и формирование отчётов.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Контрольное задание 1

Часть 1

Компания StorageManager управляет складскими запасами товаров для розничной торговли по городам России.
Ежедневно обрабатываются данные об остатках товаров на складах: часть товаров имеет достаточный запас, часть — низкий остаток, а часть — требует срочного пополнения.
Руководство хочет контролировать товары с количеством менее 15 единиц, отслеживать поставщиков и выявлять позиции с низким рейтингом поставщиков.

Ваша задача как RPA-разработчика — автоматизировать работу с данными об остатках через DataFrame в Puzzle RPA, отфильтровать товары с низким остатком, добавить аналитические столбцы, сохранить результаты и уведомить пользователя.

Шаги:

1. **Создать исходный файл inventory.xlsx с 10 записями:**

inventory.xlsx

| product_id | product_name | warehouse | quantity | category | last_update |

|---|---|---|---|---|---|

| P001 | "Ноутбук HP" | "Склад-МСК" | 25 | "электроника" | 2026-03-10 |

| P002 | "Смартфон Xiaomi" | "Склад-СПб" | 8 | "электроника" | 2026-03-09 |

| P003 | "Наушники Sony" | "Склад-МСК" | 12 | "аксессуары" | 2026-03-08 |

| P004 | "Клавиатура Logitech" | "Склад-КЗН" | 30 | "аксессуары" | 2026-03-10 |

| P005 | "Мышь Razer" | "Склад-МСК" | 5 | "аксессуары" | 2026-03-07 |

| P006 | "Монитор Dell" | "Склад-СПб" | 14 | "электроника" | 2026-03-09 |

| P007 | "SSD диск 1TB" | "Склад-МСК" | 20 | "комплектующие" | 2026-03-10 |

| P008 | "ОЗУ 16GB" | "Склад-КЗН" | 10 | "комплектующие" | 2026-03-08 |

| P009 | "Веб-камера" | "Склад-СПб" | 7 | "аксессуары" | 2026-03-09 |

| P010 | "Блок питания" | "Склад-МСК" | 18 | "комплектующие" | 2026-03-10 |

- **Типы данных:**

product_id: str

product_name: str

warehouse: str

quantity: int

category: str # "электроника", "аксессуары", "комплектующие"

last_update: date

2. **Считать исходный файл inventory.xlsx в объект в DataFrame inventory_df**

3. **Добавить столбец is_low_stock — флаг низкого остатка:**

```
inventory_df["is_low_stock"] = inventory_df["quantity"] < 15
```

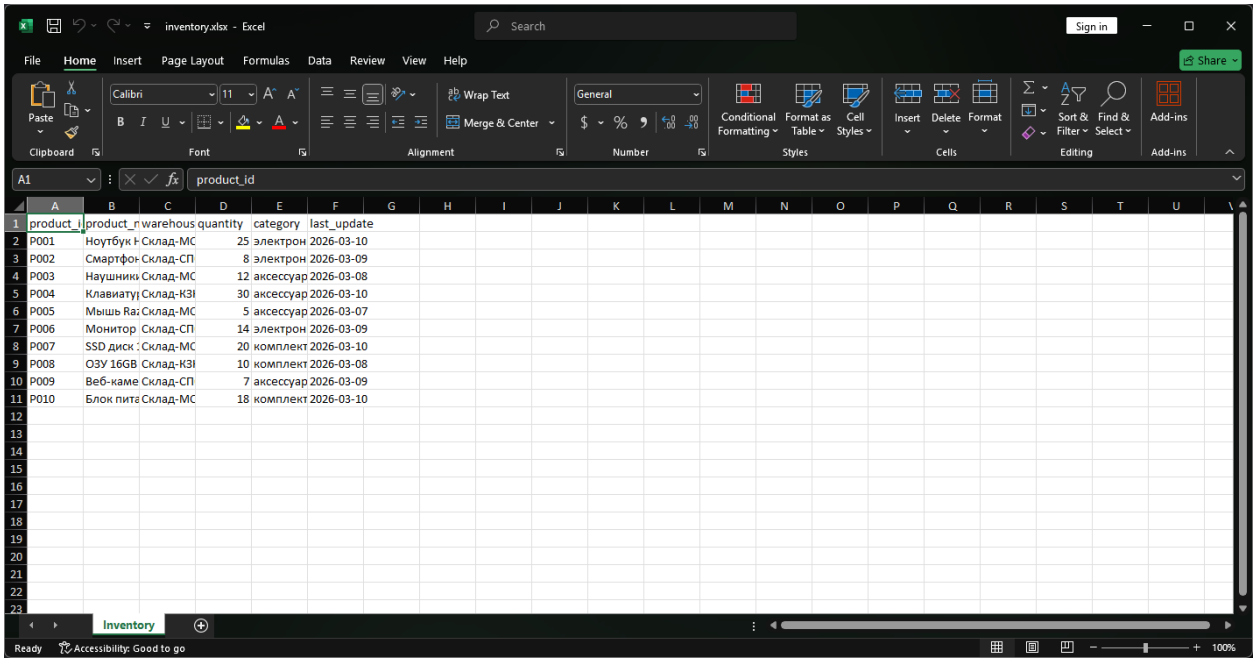
4. **Отфильтровать товары с количеством менее 15:**

```
low_stock_df = FilterDataFrame(
  table=inventory_df,
  filter_condition='quantity < 15'
)
```

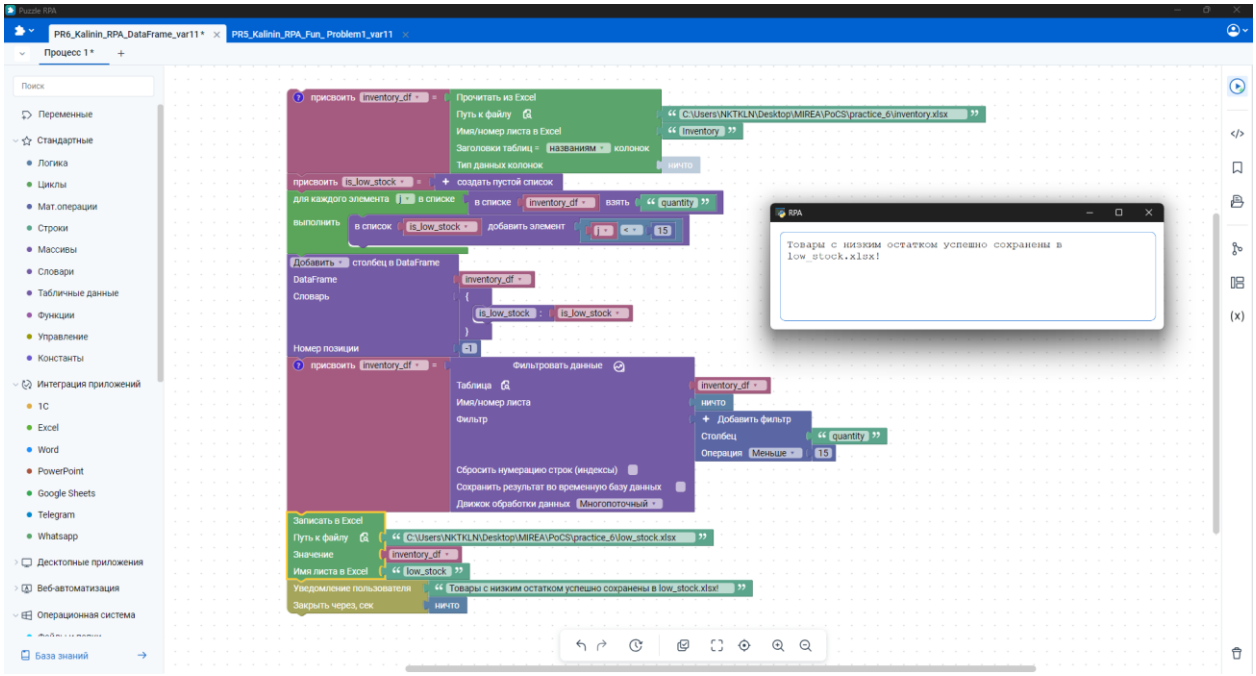
5. **Сохранить результат в Excel и уведомить пользователя:**

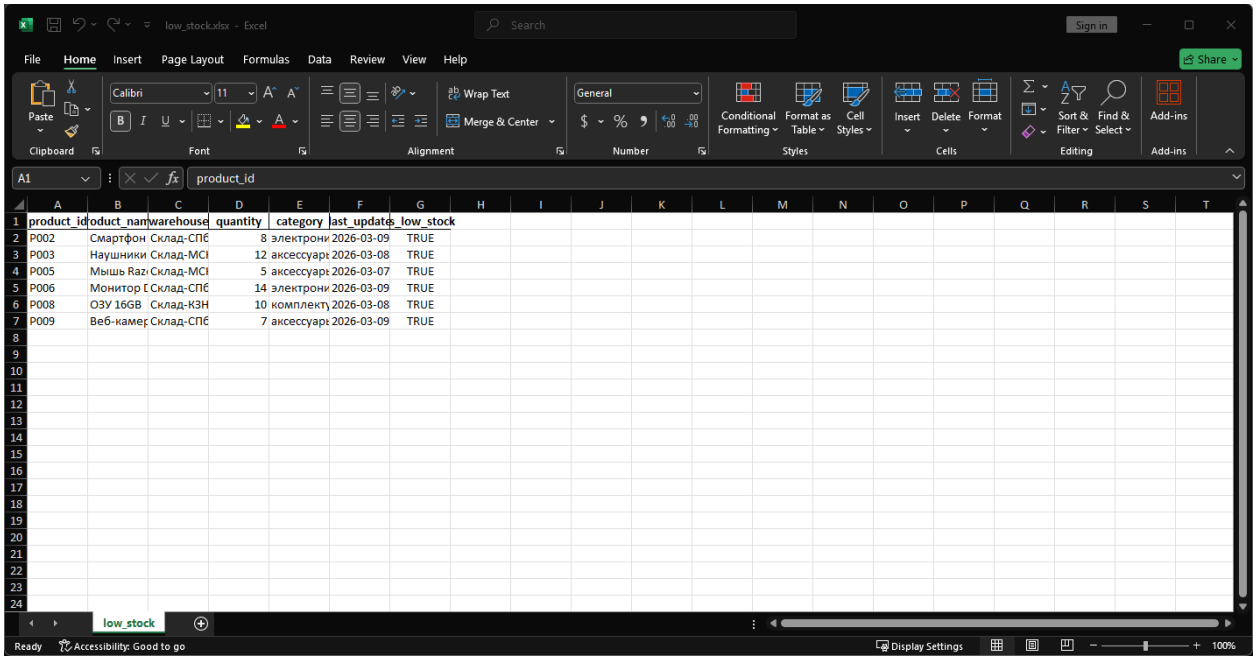
```
# Сохраняем Excel файл
SaveExcel(file_path="low_stock.xlsx")

# Уведомляем пользователя
NotifyUser(message="Товары с низким остатком успешно сохранены в low_stock.xlsx!")
```



Успешное выполнение кода процесса





Алгоритм решения: создать исходный Excel-файл inventory.xlsx, содержащий 10 записей с данными о товарах на складах в формате таблицы со столбцами product_id, product_name, warehouse, quantity, category и last_update. После этого считать файл inventory.xlsx в объект DataFrame inventory_df для дальнейшей обработки в Puzzle RPA. Затем в DataFrame добавить новый аналитический столбец is_low_stock, который показывает наличие низкого остатка по каждому товару: если значение в поле quantity меньше 15, в столбец записывается логическое значение True, иначе — False. Далее с помощью фильтрации сформировать новый DataFrame low_stock_df, включающий только те строки, в которых количество товара меньше 15 единиц. Полученный отфильтрованный результат необходимо сохранить в новый Excel-файл low_stock.xlsx. После успешного сохранения файла пользователю отправляется уведомление с сообщением о том, что товары с низким остатком успешно сохранены. Таким образом, алгоритм позволяет автоматически выделить позиции с недостаточным складским запасом, сохранить их в отдельный файл и сообщить пользователю о завершении обработки.

Контрольное задание 2

Часть 2

После фильтрации товаров с низким остатком руководство хочет проанализировать данные по поставщикам.
Существует второй файл suppliers.xlsx, в котором указаны рейтинги поставщиков и сроки поставки.
Нужно объединить данные и выделить товары, поставляемые поставщиками с рейтингом ниже 3, а результат сохранить и показать пользователю.

---

1. **Создать файл suppliers.xlsx с 8 записями:**

```
# suppliers.xlsx
| product_id | supplier_name   | supplier_rating | delivery_days |
|------------|-----------------|-----------------|---------------|
| P001       | "ООО ТехноПоставка" | 5           | 2             |
| P002       | "ИП ГаджетТрейд"    | 2           | 5             |
| P003       | "ООО ТехноПоставка" | 4           | 3             |
| P004       | "ЗАО АксессуарПлюс" | 3           | 4             |
| P005       | "ИП ГаджетТрейд"    | 2           | 6             |
| P006       | "ООО ТехноПоставка" | 5           | 2             |
| P008       | "ЗАО КомплектСервис"| 1           | 7             |
| P009       | "ИП ГаджетТрейд"    | 2           | 5             |
```

- **Типы данных:**

```
product_id: str
supplier_name: str
supplier_rating: int  # рейтинг от 1 до 5
delivery_days: int    # срок поставки в днях
```

2. **Считать исходный файл suppliers.xlsx в объект в DataFrame suppliers_df**
3. **Объединить low_stock_df и suppliers_df по product_id (left join):**

```
combined_df = JoinDataFrames(
  table1=low_stock_df,
  table2=suppliers_df,
  keys1=["product_id"],
  keys2=["product_id"],
  join_type="left"
```

)

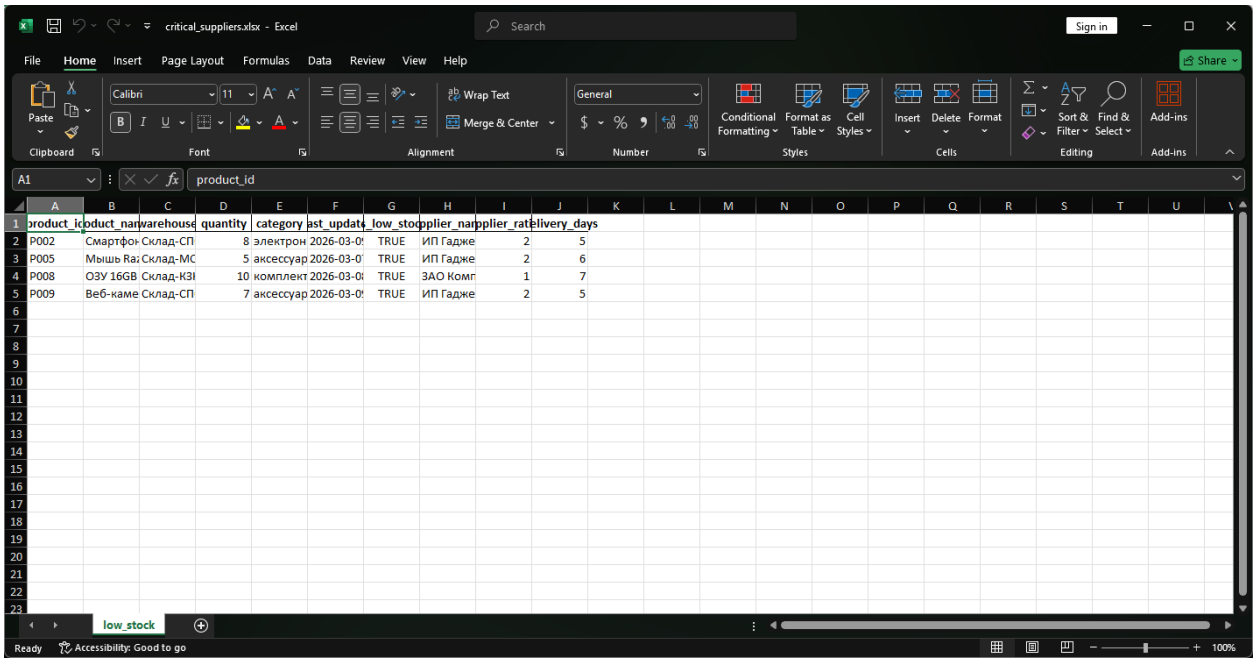
4. **Выбрать товары с рейтингом поставщика < 3:**

```
critical_suppliers_df = FilterDataFrame(
  table=combined_df,
  filter_condition='supplier_rating < 3'
)
```

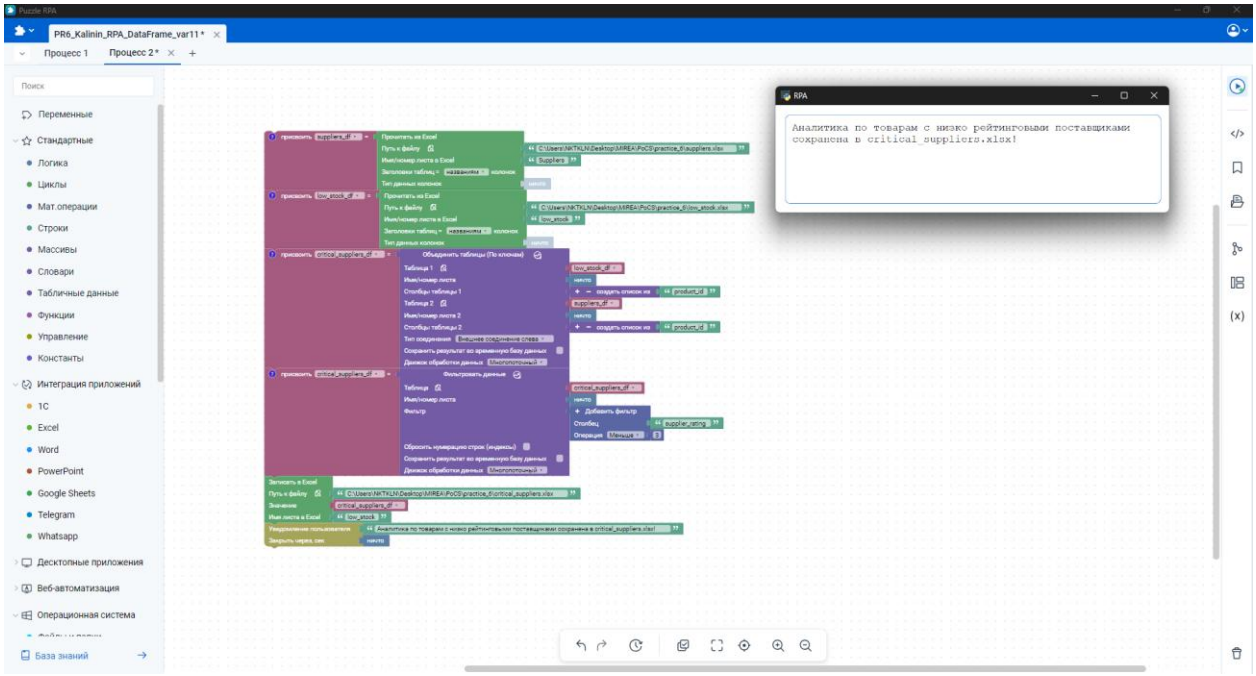
5. **Сохранить результат в Excel и уведомить пользователя:**

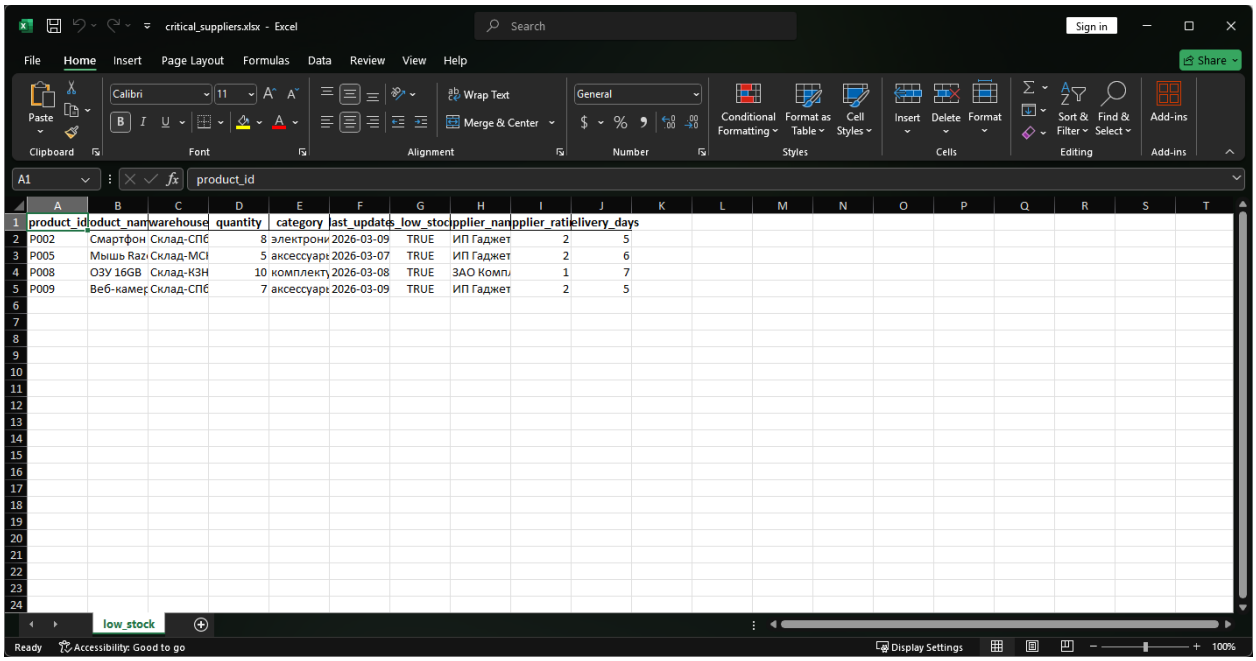
```
# Сохраняем Excel файл
SaveExcel(file_path="critical_suppliers.xlsx")

# Уведомляем пользователя
NotifyUser(message="Аналитика по товарам с низко рейтинговыми поставщиками сохранена в critical_suppliers.xlsx!")
```



Успешное выполнение кода процесса





Алгоритм решения: создать исходный Excel-файл suppliers.xlsx, содержащий 8 записей с данными о поставщиках в формате таблицы со столбцами product_id, supplier_name, supplier_rating и delivery_days. Далее считать файл suppliers.xlsx в объект DataFrame suppliers_df для последующей обработки. После этого использовать ранее полученный DataFrame low_stock_df, содержащий товары с низким остатком, и выполнить объединение двух таблиц по полю product_id с помощью операции left join, чтобы к каждому товару с низким остатком добавить информацию о поставщике. Результат объединения сохранить в DataFrame combined_df.

Затем выполнить фильтрацию объединённого DataFrame: сформировать новый DataFrame critical_suppliers_df, в который попадут только те записи, где значение supplier_rating меньше 3, что означает низкий рейтинг поставщика. После получения итогового набора данных сохранить его в Excel-файл critical_suppliers.xlsx. По завершении обработки отправить пользователю

уведомление с сообщением о том, что аналитика по товарам с низким остатком и поставщиками с низким рейтингом успешно сформирована и сохранена.

Таким образом, алгоритм позволяет автоматически объединить данные по остаткам и поставщикам, выявить проблемные позиции с точки зрения поставщиков и уведомить пользователя о готовности результата.

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_6

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/4xrsl2jhykhluj98ns